FedRAS: Rank Scheduling by Delivery Probability for Federated LoRA in Mobile Edge Networks*

Wenyu Ma[a,b,c], Xinlei Liu[a,b,c,*] and Tao Hu[a,b,c]

[a]Information Engineering University, Zhengzhou, China
[b]Key Laboratory of Cyberspace Endogenous Safety & Security of Henan Province, Zhengzhou, China
[c]Key Laboratory of Cyberspace Security, Ministry of Education of China, Zhengzhou, China

ARTICLE INFO

Keywords:
Federated LoRA
Mobile edge computing
Rank selection
Deadline scheduling
Wireless federated learning

ABSTRACT

Mobile-edge federated learning (FL) lets smartphones, vehicles, and unmanned aerial vehicles (UAVs) train models collaboratively without sharing local data, a common mobile-network workload. But full-model updates frequently miss round deadlines on degraded uplinks, and once none arrives on time, the server has nothing to aggregate and learning stops. Federated low-rank adaptation (LoRA) compresses the payload by replacing full-model updates with low-rank factors, and smaller payloads arrive on time more easily. But the payload still grows with the rank, the width of the factor pair: wider factors express more change but upload more bytes, and on a deadline-constrained link the delivery probability is lower. We propose FedRAS (federated rank-adaptive scheduling), which treats the rank as a schedulable network resource and decides jointly, under a byte budget and a participation cap, who participates and at what rank, so the update size is set by the link rather than the training side. FedRAS runs a closed round loop whose three modules score every client–rank pair, solve the assignment exactly under the byte budget, and merge heterogeneous-rank updates exactly with frozen factors. Under deadline stress, FedRAS delivers the most update mass: 22% more timely updates at 23% fewer bytes than fixed rank 16, 42% more than fixed rank 4, at statistically equivalent accuracy, while full-model updates deliver nothing. When the backbone is weak, these updates convert into accuracy, up to 1.3 percentage points over link-oblivious schedules. At 500 clients, once the budget binds, exact scheduling beats greedy at sub-second cost.

1. Introduction

Mobile edge networks increasingly host federated learning (FL) workloads. The participating devices include smartphones, vehicles, unmanned aerial vehicles (UAVs), and satellite-connected devices [1, 2], spanning Internet of Things (IoT) deployments [3] and space–air–ground integrated network (SAGIN) systems [4, 5]. Such systems train a model with the data never leaving the devices, but their links differ across devices and change with mobility, and the uplink is often the bottleneck of the whole system.

Federated training proceeds in rounds: the server broadcasts the current model, each client trains on its local data and uploads its update before a round deadline, and the server aggregates whatever arrives on time. Full-model updates—each carrying the change of every weight in the model—saturate wireless capacity and drain device energy [6], and once links degrade during handover or outage, they miss the round deadline all the more [7, 8, 9, 10]. Deadline-enforcing systems do not wait: clients that cannot finish are dropped outright for that round [11], with no buffering and no retransmission. Once the deadline tightens to the point that no full-model update completes on time, the round aggregates nothing at all: the broadcast model becomes the only model, and federated learning does not slow down—it stops.

Federated low-rank adaptation (LoRA) therefore becomes a useful technique, because each adapted layer transmits only two low-rank factors instead of the full weight delta [12, 13, 14]. But the payload still grows with the rank, and the rank is the width of this factor pair: the larger the rank, the more change the model can learn and the lower the delivery probability.

Existing federated LoRA systems treat the rank as a hyperparameter—fixed across the fleet [15, 16], varied across clients [17, 18], adapted across rounds [19, 20], or personalized through model-heterogeneous and dual-adapter designs [21, 22]. The channel-aware line of work does exploit the link state, but its optimization objective is expected latency, not the delivery probability of a specific payload within the deadline. The rank that can be delivered moves with the link—across clients, rounds, and deployment stages—so any rank fixed offline is wrong somewhere: a small fixed rank is delivered but learns too little, a large fixed rank spends bytes on uploads that never arrive, and a training-signal schedule cannot see the link at all.

Each round must therefore decide jointly, under a total byte budget and a participation cap, who participates and at what rank. Three questions follow. How should the delivery probability depend on the rank and the observable link state? How should the client–rank assignment be solved under budget and participation constraints? How should the gains of scheduling in a real trainer be measured, and when do they

*This work was supported in part by the National Key Research and Development Program of China (No. 2024YFB2907201, No. 2024YFB2907202).

*Corresponding author.
mwy_ndsc@163.com (W. Ma); liuxl@stu.ouc.edu.cn (X. Liu); hutaondsc@163.com (T. Hu)
ORCID(s): 0009-0003-9891-8062 (W. Ma); 0000-0001-6586-4457 (X. Liu); 0000-0001-7641-5622 (T. Hu)

convert into test accuracy? We invent FedRAS (federated rank-adaptive scheduling). Our contributions are threefold.

- Based on the trade-off between adaptation capacity and delivery probability—the larger the rank, the more change the model can learn and the lower the delivery probability—we design, to our knowledge, the first scheduling framework that treats the LoRA rank as a schedulable network resource and name it FedRAS. FedRAS runs a closed round loop with three modules, setting the rank from the per-round delivery state of the link rather than from training signals or expected latency. Action Construction (AC) scores every client–rank pair by its delivery probability and its adaptation capacity, with payloads priced at real serialized sizes. The GridDP assignment (GDP) decides admission and rank jointly under the byte budget and the participation cap, solving the assignment exactly on a byte grid. Masked Aggregation (MA) merges heterogeneous-rank updates exactly by frozen-factor averaging.
- Based on the multiple-choice-knapsack structure of the joint assignment, we prove that GDP solves it exactly on a byte grid (Proposition 1); that under a budget-feasibility condition the closed-form per-client rule $r_i^\star = \bar{R}_i^t \tau/(16c)$ is already optimal, enabling a zero-cost switch (Proposition 2); and that frozen-factor aggregation merges heterogeneous-rank updates exactly (Proposition 3).
- On CIFAR-10, CIFAR-100, and MNIST, using a real 20-client federated LoRA trainer and a 500-client scheduler simulation, we demonstrate the superiority of FedRAS: under deadline stress, it delivers the most update mass of the evaluated policies—22% more timely updates at 23% fewer bytes than fixed rank 16, and 42% more update mass than fixed rank 4—at statistically equivalent accuracy, while full-model updates deliver nothing; a backbone-quality sweep locates when preserved capacity converts into accuracy, up to 1.3 percentage points over link-oblivious schedules; and whenever the budget binds, exact scheduling beats greedy at sub-second cost.

2. Related Work

2.1. Federated LoRA: An Evolution Toward Adaptive Ranks

LoRA replaces the full weight delta with two low-rank factors, $\Delta W = BA$, beside a frozen backbone [12], and was ported to federated learning by sparse-update and instruction-tuning studies [13, 14]; early evaluations confirmed that parameter-efficient tuning survives the federated setting [16]. In federated LoRA the rank follows whatever information it is set from; we call that information the rank signal, and prior work reads four kinds of it.

The first kind of rank signal is an offline choice, the same for the whole fleet: the rank is a fixed hyperparameter, so the payload never responds to run-time state.

The second kind of rank signal is device capability or training-side statistics. HetLoRA prunes ranks per client and repairs the average with sparsity weights [17], FLoRA stacks factors to a common width [23], HeLoRA aligns them through padded context [18], FedARA allocates ranks by singular value decomposition (SVD) of the aggregated update [19], and a recent refinement aligns heterogeneous ranks at negligible overhead [24]. The rank now responds to the device and to training, yet not to whether this round's link can actually deliver the payload—and none of these systems prices that cost.

The third kind of rank signal is training progress. SA-FedLoRA schedules the rank over rounds by training progress [20], FedSA-LoRA couples it with asymmetric factor aggregation [25], and dual-adapter or model-heterogeneous designs personalize the adapter itself [21, 22]. Whatever the refinement, the signal remains training progress, parameter importance, or device capability; the state of the lossy, deadline-constrained link never enters the decision.

The fourth kind of rank signal, the channel itself, is the one closest to FedRAS. ROBA jointly optimizes per-client rank and bandwidth allocation under time-varying channels, with communication and computation latency in the objective [26]. TSFA selects the rank offline from statistical channel information and adapts sparsification and bandwidth online under delay constraints [27]. Adaptive federated LoRA couples sketching ratios with client sampling probabilities under bandwidth constraints to minimize wall-clock convergence time [28]. These systems do read the link, but they optimize expected latency or convergence time and treat delivery as assured once resources are granted; none prices the delivery probability of the specific payload within the round deadline.

Another line of work is concerned with how to aggregate, and FFA-LoRA is its representative. LoRA has two factors, and training usually updates both. Updating both, however, creates a problem: the same update can be written as different factor combinations, so when the server holds the factors uploaded by many clients, averaging them directly is meaningless—the factors do not align. FFA-LoRA freezes one of the two factors at initialization, originally for training stability in privacy-preserving fine-tuning [15]; its setting assumes the same rank for every client, so it never faces the question of merging different ranks. FedRAS keeps the frozen factor but applies it to the heterogeneous ranks the scheduler chooses: factor slices uploaded at different ranks land on the same coordinates, and averaging, column by column, whatever arrives on time is exact averaging of the active update slices (Proposition 3)—no alignment step, and none of the misalignment of training both factors. The scheduler can therefore assign different ranks to different clients while the aggregation layer still merges them exactly, and scheduling and aggregation meet. HetLoRA-style sparsity weighting can be read as a weighted variant of this

average, but it normalizes with one common denominator, so high-rank slices are diluted by low-rank clients; FedRAS normalizes each column by the clients that actually cover it, keeps the high-rank information intact, and this is exactly where the exactness of its aggregation comes from.

2.2. Communication-Efficient Federated Learning

FedRAS and compression do not solve the same problem. Compression decides how to transmit a given object in fewer bits; FedRAS decides how large an object to transmit over a given link. Sparsification [29], quantization [30, 31], quantized compressed sensing [32, 33], and over-the-air aggregation [34] do compress the payload, and codec adaptation tracks channel state. But no codec decides the size of the object it encodes, and compression cannot recover a deadline that an already-oversized payload is destined to miss. FedRAS selects the rank before transmission, swapping a payload destined to miss this round for a smaller one that arrives on time, and so preserves the update mass. Compression, in turn, can only shrink the payload after it is chosen; it cannot change whether that payload arrives. Compression acts on the payload FedRAS has chosen: the two work at different stages, compose freely, and do not replace each other. Compression and scheduling can therefore be used together—one decides how large to transmit, the other how to transmit it in fewer bits.

2.3. Client Selection and Deadline-Aware Scheduling

Client selection decides who participates—but in all of this work the payload size is an input, never a decision. Oort ranks candidates by a utility that couples data value with system efficiency [35], FedCS admits clients under explicit resource constraints [11], and TiFL bounds stragglers through latency tiers [36]; further designs target heterogeneity, cost, and freshness [37, 38, 39]. Deadline- and reliability-aware schedulers extend admission to mobile and non-terrestrial networks, from participation under handover and outage [7, 8, 10, 40] to offloading driven by federated reinforcement learning [9], with LESSON admitting wireless clients at differentiated frequencies under a latency budget [41] and FedSN coordinating training over heterogeneous low-Earth-orbit (LEO) constellations [42]. These systems fold the deadline into admission, but the payload remains given, so the choice is only among updates of a fixed size. Such a system has two options—transmit an oversized payload that is destined not to arrive, or skip the update entirely—because it cannot break the payload down to a size the link can carry. Once the payload itself becomes a decision, the problem changes in nature: every client carries several rank options, each with its own bytes and gains, and the choice is no longer a ranking or a threshold but the selection of a combination under the byte budget and the participation cap, in which greedy rules are not necessarily optimal (Propositions 1 and 2). The advantage of FedRAS is that it merges selection and payload into one decision, using exact scheduling to decide who participates and at what rank; an update that could previously only be skipped or forced through can be shrunk to a size the link delivers on time, so more timely updates are delivered under the same budget and deadline. This is why FedRAS needs a new solver rather than a rank parameter added to an existing selector, and why an Oort-style selector can here serve only as the baseline: it cannot handle the case where each client carries several payload options.

2.4. Research Gap and Positioning

Across the surveyed lines, the update size is fixed by the training side: rank methods read training progress or device capability, the wireless-aware line reads the channel but prices expected latency, and selection methods schedule a payload they did not choose. None treats the LoRA rank as a schedulable network resource priced by the payload's delivery probability, and none solves admission and rank jointly under a byte budget and a deadline. FedRAS fills exactly this gap: the rank is priced at its real serialized size, conditioned on the per-round delivery state of the link, and decided jointly with admission. Table 1 positions FedRAS against the closest systems; throughout the table, the delivery-aware column is the watershed, and the table note defines it—conditioning on the per-round payload delivery probability, not merely reading the channel, which is why ROBA and TSFA remain ×.

3. System Model and Threat Model

3.1. Mobile-Edge Federated Learning Architecture and Round Protocol

We consider a mobile edge network in which an edge aggregator serves a client set $\mathcal{D} = \mathcal{D}_g \cup \mathcal{D}_a \cup \mathcal{D}_s$ of ground clients $\mathcal{D}_g$, aerial and UAV clients $\mathcal{D}_a$, and satellite-connected clients $\mathcal{D}_s$. Ground links are comparatively stable, aerial links face frequent handover, and satellite links suffer long propagation delay, intermittent visibility, and outages [5, 7]. The aggregator holds the global model, schedules every round, and aggregates the timely updates. This network is also the running example followed throughout the paper. The participants are the client fleet, one aggregator per region, and a passive uplink eavesdropper. The service flow repeats each round. The server, which has centrally pretrained the backbone on its own corpus, broadcasts the frozen backbone and the shared factor and attaches the round schedule. Scheduled clients fine-tune their factor slices on private local data, which never leaves the devices, and upload before the deadline. The server returns the masked aggregate of whatever arrived on time. Disclosure is controlled by the aggregator alone, through the admission and rank decisions, since a client transmits exactly the coordinates of its assigned rank. What the eavesdropper can reconstruct from an interception is therefore the induced update of that rank, the weight change its transmitted coordinates produce, and nothing beyond it. Whether the transmitted coordinates reveal training information is a question FedRAS does not claim to answer, and smaller ranks strictly reduce the exposed bytes and coordinates.

Table 1
Positioning against federated LoRA and client-selection systems. ✓ = the property holds in the cited papers' own descriptions; × = absent or not reported. Delivery-aware means that a scheduling or adaptation decision conditions on per-round network delivery state (rate, deadline, delivery probability) rather than on training progress, device capability, or static profiling alone; the wireless-aware rank line (ROBA, TSFA, adaptive FL) reads channel state but optimizes expected latency or convergence time with delivery assumed once resources are granted, so it is not delivery-aware in this per-payload sense. Real payload means payload is accounted as transmitted bytes rather than a fixed or parameter-count proxy. Real training means model training is executed in the evaluation as reported.

System	Var. rank	Deliv.-aware	Het.-rank agg.	Admission	Joint adm.+rank	Real payload	Real training
FFA-LoRA [15]	×	×	×	×	×	×	✓
SLoRA [13]	×	×	×	×	×	×	✓
FedDPA [22]	×	×	×	×	×	×	✓
HetLoRA [17]	✓	×	✓	×	×	×	✓
HeLoRA [18]	✓	×	✓	×	×	×	✓
FedARA [19]	✓	×	✓	×	×	×	✓
SA-FedLoRA [20]	✓	×	×	×	×	×	✓
FedSA-LoRA [25]	×	×	×	×	×	×	✓
ROBA [26]	✓	×	×	×	×	×	×
TSFA [27]	✓	×	×	×	×	×	×
Adapt. FL [28]	✓	×	×	✓	×	×	×
Oort [35]	×	×	×	✓	×	×	✓
FedCS [11]	×	✓	×	✓	×	×	✓
TiFL [36]	×	×	×	✓	×	×	✓
LESSON [41]	×	✓	×	✓	×	×	✓
FedSN [42]	×	✓	×	✓	×	×	✓
FedRAS (ours)	✓	✓	✓	✓	✓	✓	✓

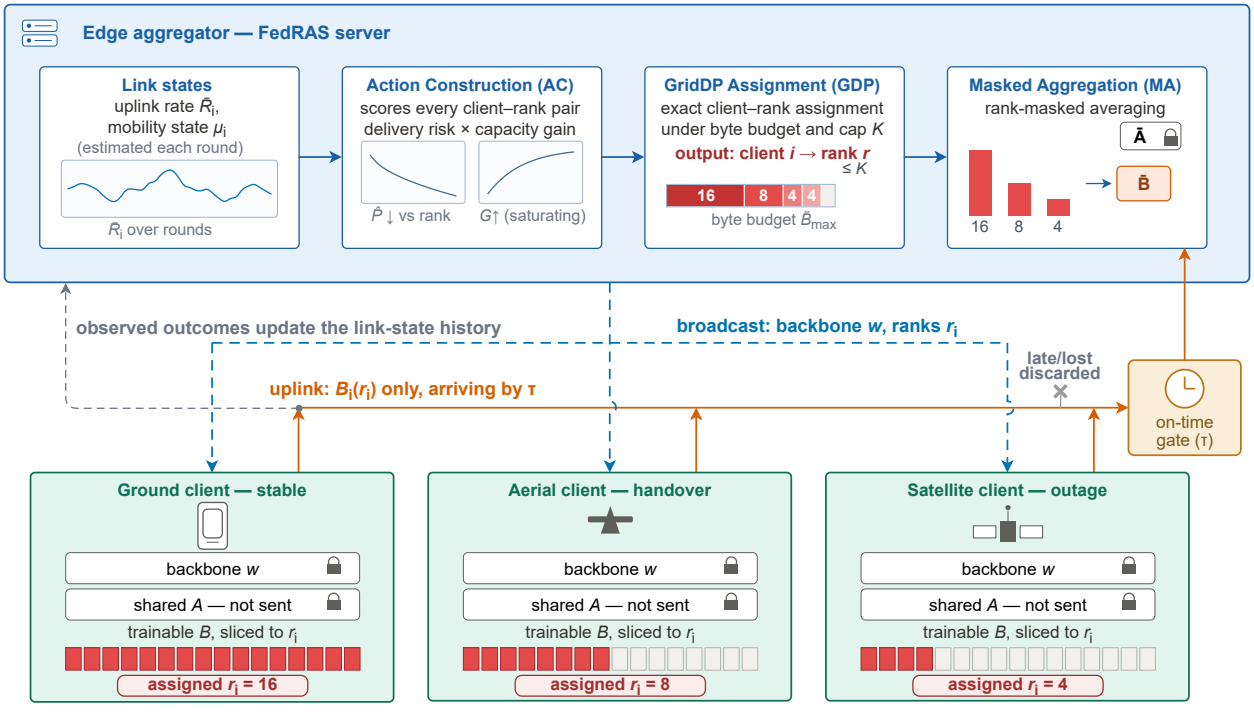


Figure 1: The FedRAS architecture. Action Construction (AC) scores every client–rank pair with the hazard model (3) and the saturating capacity (5); the GridDP assignment (GDP) computes the client–rank assignment under the byte budget and the participation cap. Updates arriving within the deadline τ pass the on-time gate and are merged by Masked Aggregation (MA) while the shared factor A stays frozen; observed outcomes update the link-state history. AC, GDP, and MA correspond to Algorithms 2 to 4; the overall round is Algorithm 1.

Let w^t denote the frozen backbone broadcast at round t. Client i trains LoRA factors $A_{i,\ell}$ and $B_{i,\ell}$ for the adapted layers $\ell \in \mathcal{L}$, with effective weight change $\Delta W_{i,\ell} = B_{i,\ell} A_{i,\ell}$. A rank r update uses the first r columns of $B_{i,\ell}$ and the matching rows of the shared A. Figure 1 shows the resulting architecture. In every round, the aggregator estimates the link states, scores the client–rank actions with Action Construction (AC), and solves the assignment with the GridDP assignment (GDP); the admitted clients train locally with their factors sliced to the assigned rank, so only the first r coordinates receive gradient updates, and upload before the deadline τ; the aggregator then merges the timely updates with Masked Aggregation (MA) and updates its link-state history from the observed outcomes. Late or lost updates are discarded, because stale updates harm convergence stability in deadline-critical federated learning. The experiments evaluate both a homogeneous link profile and a grouped ground–aerial–satellite profile with distinct rate and mobility laws.

3.2. Threat Model

Adversary and capabilities. The external adversary is a passive eavesdropper on the wireless uplink: it observes the B factors a client transmits, the public frozen factor A, and the round schedule, and nothing else, it cannot modify or drop messages (integrity and availability belong to the link model), cannot observe local data, and mounts no active attack.

Trust, assumptions, and boundary. Clients are honest but curious, they follow the training protocol, and raw data never leaves the devices, the standing premise of federated learning on this infrastructure. The edge aggregator is trusted scheduling infrastructure: it observes link states, client–rank assignments, and delivered update factors, and runs the masked aggregation. Delivery outcomes follow the link model of the next section; clients report nothing beyond what any FL round makes observable.

Protection target. Because A is public and a rank-r client transmits only its first r coordinates, the object reconstructable from an intercepted transmission is the rank-r induced update, and the exposed surface is its B-slice: strictly smaller in bytes and in exposed coordinate count than a joint-factor or full-model upload. This reduction in exposed transmission is a communication property, and it is the only security-relevant claim made here: defending the transmission content itself, together with differential-privacy accounting, secure aggregation, and Byzantine robustness, is out of scope and composes orthogonally with rank scheduling. Every method and experiment with FedRAS stays inside this boundary; no attack is mounted and no defense is evaluated.

3.3. Link, Mobility, and Timely-Delivery Model

At decision time the scheduler receives an estimated uplink rate $\bar{R}_i^t$ in Mbps and a mobility state. The realized rate R_i^t, handover event $H_i^t \in \{0, 1\}$, and packet loss indicator J_i^t are not observed before transmission. For an update of $B_i^t(r)$ megabytes, the modeled delivery time is

$$T_i^t(r) = T_{\text{train,i}}(r) + T_{\text{proc,i}}(r) + \frac{8B_i^t(r)}{R_i^t} + d_i^t + H_i^t \Delta d_h. \quad (1)$$

Timely delivery is the event

$$P_{\text{on,i}}^t(r) = \Pr\left(T_i^t(r) \le \tau,\ J_i^t = 0 \mid x_i^t\right). \quad (2)$$

Deriving an analytical delivery-time distribution for every term of (1) under real mobility and handover dynamics is non-trivial. Two conventions make the model operational. First, τ is the uplink delivery window that opens when local training finishes: the deadline binds the transmission, propagation, and handover terms, and every experiment interprets it this way. Second, the rank-dependence of T_{train} and T_{proc} is absorbed, as a stated approximation, into the rank-independent intensity μ_i^t; for the evaluated backbones the adapters hold at most a few percent of the parameters, so the rank-dependence of local compute is second-order, and the setup reports the measured local training time. The product form assumes the loss event and the delay event are independent; at the nominal calibration the mobility factor dominates the ratio while at the stress calibrations the transmission factor does, which keeps the two effects separable in the analysis. We therefore adopt the surrogate hazard model, a closed-form failure-risk factor in the survival-analysis sense standing in for the event (2),

$$\hat{P}_{\text{on,i}}^t(r) = \exp\left(-\mu_i^t\right) \exp\left(-\frac{8B_i^t(r)}{\bar{R}_i^t \tau}\right). \quad (3)$$

The dimensionless intensity μ_i^t summarizes mobility-related failure risk, packet loss, handover effects, and unmodeled processing time. Equation (3) therefore serves as a closed-form plug-in predictor, and the sensitivity and calibration studies quantify how prediction imperfections degrade the scheduling objective.

3.4. Rank-Dependent Payload and Adaptation Capacity

With b bits per parameter and the frozen factor A shared and never transmitted, a rank-r client uploads only the trainable factor B, whose per-rank cost is the summed output dimension $D = \sum_{\ell \in \mathcal{L}} d_\ell$ of the adapted layers. The real serialized payload is

$$B_i^t(r) = \frac{b}{8} r D + B_{\text{meta}}, \quad (4)$$

in bytes. Rank is therefore a schedulable network resource: it scales both the adaptation capacity and the transmission time, and its per-unit payload is audited against the evaluated model. We model the useful contribution of a delivered rank-r update with the saturating capacity gain

$$G_i^t(r) = \pi_i^t \sqrt{\min(r, r_{\max})/r_{\max}}, \quad (5)$$

where π_i^t is a per-client contribution prior in the spirit of Oort-style utility [35]. The width $r_{\max}$ is the factor width provisioned at all clients. The square root encodes the diminishing returns the delivered-update norms exhibit in the trainer,

with saturation at the provisioned width; that the norm is a faithful proxy for training contribution is the underlying modeling assumption, which the sensitivity analysis bounds by sweeping the exponent. Full-model FedAvg updates [1] are accounted at their true serialized size (Table 2).

3.5. Deadline-Constrained Client–Rank Scheduling Formulation

Each round, the aggregator decides $a_i^t \in \{0\} \cup \mathcal{R}$, where $a_i^t = 0$ means client i does not participate and $a_i^t = r$ schedules it at rank r. The per-round problem is

$$\max_{\{a_i^t\}} \sum_{i \in \mathcal{D}} \mathbf{1}\{a_i^t > 0\}\, G_i^t(a_i^t)\, P_{\mathrm{on,i}}^t(a_i^t) \tag{6}$$

$$\text{s.t.} \quad \sum_i B_i^t(a_i^t) \le B_{\max}, \qquad \sum_i \mathbf{1}\{a_i^t > 0\} \le K. \tag{7}$$

The objective is a scheduling utility: the expected amount of useful adaptation capacity delivered on time in the round; $B_{\max}$ is the per-round byte budget and K the participation cap. Increasing r raises G sublinearly while lowering the delivery term, so the two effects are priced in one common unit; we call the realized quantity—the update mass actually aggregated on time, measurable in the trainer from delivered update norms—the *delivered adaptation capacity*, and it serves as the single learning-side currency of the evaluation. The utility is not a surrogate for test accuracy, and its relation to accuracy is established empirically rather than assumed. When the frozen backbone is already accurate, rank choice and final accuracy are equivalent within a small pre-fixed margin, which the evaluation verifies with a two one-sided tests (TOST) procedure; this invariance is not a defect of the formulation but its enabling condition, since exactly because accuracy does not depend on the rank, the rank is free to be set by the link. When the backbone is weak enough to need adaptation, the capacity this utility maximizes converts into measured accuracy: a backbone-quality sweep characterizes the conversion, and the evaluation reports tasks where federated communication buys roughly three accuracy points over the frozen backbone itself, about one point when the pretraining and deployment input distributions differ, and a collapse to the minimum rank when the capacity term is removed. The experiments therefore report accuracy and the delivered-capacity outcomes as separate metrics, and the correspondence between them is carried by the capacity term.

4. FedRAS Design

4.1. Overview

Algorithm 1 summarizes one round and Figure 1 places the stages. The three design questions of Section 1 map one-to-one onto the three modules. Action Construction (AC) answers how the rank should be priced: it scores every client–rank pair with a real payload, a delivery probability, and a capacity gain, so the exchange between capacity and delivery is settled before any decision is made. The GridDP assignment (GDP) answers how admission and rank should be decided together: it solves the scored assignment exactly under the byte budget and the participation cap, with a cheap per-client rule covering the unconstrained regime. Masked Aggregation (MA) answers how heterogeneous-rank updates can be merged without error: it averages the delivered factor slices exactly toward the frozen shared factor. The design contains no training-side logic: admissions and ranks are chosen from network state alone.

Algorithm 1 FedRAS Round

Input: clients $\mathcal{D}$, rank set $\mathcal{R}$, cap K, budget $B_{\max}$, deadline τ, link-state history $\mathcal{H}^{t-1}$

Output: assignment a^t; new adapter state S^t; updated history $\mathcal{H}^t$

1. Estimate per-client link states x_i^t from $\mathcal{H}^{t-1}$
2. Score all pairs (i, r): utilities $u_i(r)$ and integer costs $\widetilde{B}_i(r)$ (AC, Algorithm 2)
3. Solve the assignment a^t under K and $B_{\max}$ (GDP, Algorithm 3); when the all-$r_{\max}$ profile is budget-feasible, the per-client maximizer rule is already optimal (Proposition 2)
4. Broadcast the backbone w^t; admitted clients train locally with factors sliced to a_i^t and upload before τ
5. Pass uploads arriving within τ (on-time gate)
6. Aggregate timely updates with the frozen shared A (MA, Algorithm 4)
7. Append observed outcomes (timely, late, lost) to $\mathcal{H}^t$

4.2. Action Construction (AC)

Each round the aggregator selects, for every client, one action from the discrete set $a_i^t \in \{0\} \cup \mathcal{R}$ of the formulation. AC builds and scores this action set (Algorithm 2): the per-rank payload follows (4), the hazard model (3) and the capacity gain (5) give the estimated utility $u_i(r) = G_i^t(r)\hat{P}_{\mathrm{on,i}}^t(r)$, and a conservative discretization onto a budget grid q yields the integer cost $\widetilde{B}_i(r) = \lceil B_i^t(r)/q \rceil$ and the integer budget cap $\widetilde{B}_{\max} = \lfloor B_{\max}/q \rfloor$: rounded-up costs never understate a bill and a rounded-down cap never overstates the budget, so every grid-feasible plan is feasible in true bytes. The same grid is applied to the dynamic program, the greedy baselines, and the admission checks, so that every scheduling method optimizes over the identical feasible region; under it the estimated problem becomes a multiple-choice knapsack with a participation constraint, solvable exactly by a dynamic program (DP) whose cost grows with the numeric budget (Proposition 1); the solver study reports the evaluated grid width and the induced quantization error.

4.3. GridDP Assignment (GDP)

GDP solves the scored assignment exactly (Algorithm 3). Let $V_i(k, b)$ be the maximum estimated utility from the first i clients with k admissions and integer budget b. Initialize

Algorithm 2 Action Construction (AC)

Input: clients $\mathcal{D}$, ranks $\mathcal{R}$, budget grid q, link states $\{x_i^t\}$
Output: scored action set $\{u_i(r),\ \widetilde{B}_i(r)\}_{i\in\mathcal{D},\, r\in\mathcal{R}}$

for each client i and rank r **do**
  Payload $B_i^t(r)$ by (4); integer cost $\widetilde{B}_i(r) \leftarrow \lceil B_i^t(r)/q \rceil$
  Delivery probability $\hat{P}^t_{\mathrm{on,i}}(r)$ by (3); capacity $G_i^t(r)$ by (5)
  Utility $u_i(r) \leftarrow G_i^t(r)\, \hat{P}^t_{\mathrm{on,i}}(r)$
return the scored set; pairs with $\widetilde{B}_i(r) > \widetilde{B}_{\max}$ are infeasible

Algorithm 3 GridDP Assignment (GDP)

Input: scored actions $\{u_i(r),\ \widetilde{B}_i(r)\}$ from AC, cap K, budget grid $\widetilde{B}_{\max}$
Output: client–rank assignment a^t

$V_0(0,0) \leftarrow 0$; all other DP states $\leftarrow -\infty$
for each client $i \in \mathcal{D}$ **do**
  for $k = 0, \dots, K$ and each budget state b **do**
    $V_i(k,b) \leftarrow$ skip or best rank transition of (8)
    Store the maximizing action as a backpointer
Backtrack from $\max_{k,b} V_{|\mathcal{D}|}(k,b)$; return a^t

$V_0(0,0) = 0$ and all other states to $-\infty$. The recurrence is

$$V_i(k,b) = \max\Big\{ V_{i-1}(k,b),\ \max_{r\in\mathcal{R}} \big[V_{i-1}(k-1, b-\widetilde{B}_i(r)) + u_i(r) \big] \Big\}, \tag{8}$$

where $u_i(r) = G_i^t(r)\hat{P}^t_{\mathrm{on,i}}(r)$. Backpointers recover the assignment. Propositions 1 and 2 establish the exactness of this recurrence and the regime in which the per-client maximizer rule $r_i^\star$ is already optimal, in which case GDP reduces to that rule at zero cost.

4.4. Masked Aggregation (MA)

Clients scheduled at different ranks share an $r_{\max}$ coordinate system. For one adapted layer, client i has sample weight n_i and delivered rank r_i. The factor $A \in \mathbb{R}^{r_{\max}\times k}$ is frozen and shared, while $B_i \in \mathbb{R}^{d\times r_{\max}}$ is trainable. A rank r_i update activates columns 1 through r_i of B_i and the matching rows of A, giving $\Delta W_i = \sum_{j\le r_i} [B_i]_{:,j}[A]_{j,:}$.

Jointly trained factor pairs (A_i, B_i) are identifiable only up to invertible transforms, so separately averaged factors need not average the updates at all; the elevated-learning-rate ablation revisits this contrast empirically.

Rank-masked averaging. The server zero-pads inactive columns of each delivered B_i and averages each active coordinate over the clients that activate it,

$$\bar{B}_{:,j} = \frac{1}{W_j} \sum_{i:\, r_i \ge j} n_i \big[B_i\big]_{:,j}, \qquad W_j = \sum_{i:\, r_i\ge j} n_i, \tag{9}$$

while the frozen factor aggregates to itself, $\bar{A} = A$. The server then applies momentum per factor tensor. With $\Delta^t = \bar{B} - S^{t-1}$, the velocity and state update as $v^t = \beta\, v^{t-1} + (1-\beta)\,\Delta^t$ and $S^t = S^{t-1} + v^t$, with damping coefficient β absorbing client-sampling and link noise. Two edge rules fix the degenerate cases: a round with no timely update is a no-op (state and velocity carry over unchanged), and a rank column with no delivered coverage ($W_j = 0$, which arises when every delivered rank is below j) follows the configured fallback, the momentum target pulls it to zero, or the previous state is held; the evaluation defaults to the former and the robustness study ablates the choice. Algorithm 4 fixes the procedure; Proposition 3 states its exactness and dilution properties.

Algorithm 4 Masked Aggregation (MA)

Input: timely updates $\{(n_i, B_i, r_i)\}$, frozen shared A, momentum β, previous state S^{t-1}
Output: new adapter state S^t

for each rank column j of each adapted layer **do**
  $W_j \leftarrow \sum_{i:\, r_i\ge j} n_i; \quad \bar{B}_{:,j} \leftarrow \frac{1}{W_j} \sum_{i:\, r_i\ge j} n_i\, [B_i]_{:,j}$
Empty round (no timely update): **return** S^{t-1} with $v^t \leftarrow v^{t-1}$ (state and momentum carry over)
Columns with $W_j = 0$: apply the configured fallback (pull to zero through the momentum target, or hold S^{t-1}; both variants are ablated)
$\bar{A} \leftarrow A$
$\Delta^t \leftarrow \bar{B} - S^{t-1}; \quad v^t \leftarrow \beta\, v^{t-1} + (1-\beta)\,\Delta^t; \quad S^t \leftarrow S^{t-1} + v^t$

5. Theoretical Analysis

5.1. Exactness of the DP Assignment

Proposition 1 (Per-round optimality)**.** *For every reachable state (k,b), recurrence (8) considers the skip action and every feasible rank for client i. Induction on i therefore proves exactness for the discretized single round estimated assignment problem.*

Proof. The admission indicator and the rank of each client are exclusive actions that affect the two resources additively. Admitting client i at rank r consumes one admission slot and $\widetilde{B}_i(r)$ budget units and adds $u_i(r)$ value, independent of the other clients. The state (k,b) therefore summarizes all interaction, and the standard principle-of-optimality induction for the multiple-choice knapsack applies. Any optimal solution restricted to the first $i-1$ clients must itself be optimal for its state, which the recurrence maximizes. Utilities are nonnegative by construction, so the final maximization over (k,b) is well defined. □

Exactness is relative to the discretized problem: any residual gap to the continuous optimum is quantization error, controlled by the grid step and measured by the solver study. It also localizes loss: given the estimated probabilities, no feasible schedule outscores the DP output, so any planned-versus-realized gap originates in prediction error or the round-level abstraction, not the solver, which is what makes the calibration study interpretable.

5.2. Optimality Condition of the Per-Client Rule

The formulation also yields a condition under which the cheap per-client rule, admit every candidate and assign each its own maximizer $r_i^\star = \arg\max_{r\in\mathcal{R}} G_i(r)\hat{P}_{\text{on},i}(r)$, is already optimal, which the scheduling-arm experiments rely on.

Proposition 2 (Cohort-local optimality of the per-client rule)**.** *Consider the single-round problem (6) and (7) restricted to a fixed candidate set of at most K clients, and suppose the all-$r_{\max}$ rank profile is budget-feasible, i.e., $\sum_i \widetilde{B}_i(r_{\max}) \le \widetilde{B}_{\max}$ over the candidates. Then the profile that assigns every admitted client its per-client maximizer $r_i^\star = \arg\max_{r\in\mathcal{R}} G_i(r)\hat{P}_{\text{on},i}(r)$ is a feasible optimal solution of the restricted problem, unique whenever the per-client maximizers are.*

Proof. Each candidate's cost is at most its $r_{\max}$ cost, so the per-client maximizer profile satisfies the budget whenever the all-$r_{\max}$ profile does, and, with at most K candidates, it uses no more than the cap. Being feasible, it attains $\sum_i \max_r u_i(r)$, which upper-bounds the objective of every feasible profile; it is therefore optimal. By Proposition 1, the dynamic program returns an optimal profile and hence coincides with the per-client maximizer profile whenever the latter is unique. □

The condition has a direct economic reading: Proposition 2 holds when the budget is not scarce enough to force a sacrifice, that is, when every candidate can take its unconstrained favorite rank. In that regime the objective decomposes across clients, local selfishness is globally optimal, and the scheduler reduces to $|\mathcal{D}|$ table lookups. Greedy rules can fail only when this condition breaks: suboptimality requires the budget or the cap to force trade-offs between clients, and whether that occurs is exactly what the solver study quantifies. Once the budget forces trade-offs, the action that is best for one client changes the residual budget available to the others, and no per-client ordering resolves that coupling. The condition is also actionable at run time: checking $\sum_i \widetilde{B}_i(r_{\max}) \le \widetilde{B}_{\max}$ costs one pass over the candidates, so the per-client rule runs by default and the dynamic program is invoked only when the check fails or the pool must be culled, precisely where admission coupling appears.

Remark (closed-form per-client rank). With the linear payload (4) and the hazard (3), the per-client utility is $u_i(r) \propto \sqrt{r/r_{\max}}\,\exp\!\left(-8cr/(\bar{R}_i^t\tau)\right)$ with c the per-rank byte cost, so the continuous relaxation is maximized at

$$r_i^\star = \frac{\bar{R}_i^t\tau}{16c}, \tag{10}$$

the bytes deliverable within the deadline over sixteen per-rank costs, evaluated at the nearest candidate rank. Two consequences follow. First, the per-client rank is set by rate and deadline alone: the mobility factor $e^{-\mu_i^t}$ cancels in the argmax, so mobility shapes which clients are worth admitting and the utility level of an admission, but never shifts the rank of an admitted client, the mobility-only scenarios of the robustness study return exactly this algebraically forced parity. Second, on slow links the optimum is small even when the budget is entirely slack: $\bar{R}_i^t = 2\,\text{Mbps}$ with $\tau = 0.1\,\text{s}$ gives $r_i^\star \approx 2.2$, candidate rank 2. Downshifting under stress is therefore the rule's designed response to the link, not an artifact of scarcity.

5.3. Exactness of Frozen-Factor Aggregation

Proposition 3 (Exactness of frozen-factor aggregation)**.** *Under (9), the j-th active rank slice of $\bar{B}A$ is the data-weighted average over clients with $r_i \ge j$. This is exact coordinate-wise averaging of active induced-update slices. It is not, in general, the unified data-weighted average of complete client updates. If a common denominator $W = \sum_i n_i$ is used instead of W_j, the unified average is recovered, but active high-rank slices are diluted by W_j/W. The two rules coincide for all possible updates when all delivered ranks are equal.*

Proof. The first two parts follow from linearity. For frozen shared A, multiplying a weighted sum of B-coordinates by $\left[A\right]_{j,:}$ converts it into the same weighted sum of the matching update slices, and the two normalizers differ only in W_j versus W. The product $\bar{B}A = \sum_j \bar{B}_{:,j}\left[A\right]_{j,:}$ collects the slices. The third part holds because $W_j = W$ for every active j if and only if $r_i = \max_k r_k$ for all admitted i with $n_i > 0$. □

The frozen factor is what makes heterogeneous ranks commensurable. If both factors rotate per client, the product structure of an update is client-specific and no server-side average of factors corresponds to an average of updates; with A shared, rank slicing acts on B alone and the j-th slice of every delivered update lives in the same coordinates. The two normalizers then embody a genuine choice, per-slice fidelity (W_j) versus the FedAvg-style unified average (W, which dilutes high-rank slices by W_j/W); Proposition 3 states the trade and the coincidence of the two rules when delivered ranks do not mix.

5.4. Complexity Analysis

The complexity is $O(|\mathcal{D}|K\widetilde{B}_{\max}|\mathcal{R}|)$ time and $O(K\widetilde{B}_{\max})$ memory with rolling value arrays; the back-pointer table retained for reconstruction adds $O(|\mathcal{D}|K\widetilde{B}_{\max})$, where $\widetilde{B}_{\max} = \lfloor B_{\max}/q \rfloor$. The solver study reports the resulting runtimes at 500- and 1000-client scale.

6. Experimental Evaluation

The evaluation proceeds in seven stages: the setup; the main experiment in the real trainer (the deadline ladder and its delivered-capacity ranking); the adaptive-task tiers and the backbone-quality sweep that locate where capacity converts into accuracy; the comparison against baselines; the ablation and sensitivity analysis; the efficiency and robustness studies; and the scale and solver study. All experiments stay inside the threat boundary of Section 3.2: the adversary is the passive eavesdropper defined there, and every payload

and delivery outcome refers to the link model of Section 3. All byte figures are real serialized sizes, and all deadline regimes are calibrated to those sizes.

6.1. Experimental Setup

Environment. The trainer is implemented in PyTorch, and all real-training runs use one NVIDIA A800 graphics processing unit (GPU).

Datasets and partitions. Experiments use CIFAR-10, CIFAR-100, and MNIST. The real trainer partitions 20 clients non-IID (non-independent, non-identically distributed client label distributions) with a Dirichlet split of concentration $\alpha = 0.5$ and samples 10 participants per round; variations of α and of the mobility distribution are reported in the extended stress matrix. A distribution-shift variant of CIFAR-10 separates the pretraining distribution from the deployment distribution: the backbone pretrains centrally on clean images, while the client corpus and the test set apply a deterministic covariate shift (a change of the input distribution: additive Gaussian noise and a brightness/contrast offset of severity s), fixed per image so the shifted corpus is a dataset rather than an augmentation. With *disjoint pretraining* the backbone sees only half of the training images and the clients hold a corrupted view of the disjoint complement, so no client image was seen in pretraining; the default variant pretrains on the full clean set, and both are reported.

Backbone and adapters. The backbone is a six-layer convolutional neural network (CNN) with BatchNorm with channel progression 64/128/256, followed by two 256-wide linear layers. It is centrally pretrained with crop, flip, and erase augmentation, Nesterov momentum, weight decay, and cosine learning-rate decay, for 30 epochs on CIFAR-10, 45 on CIFAR-100, and 3 on MNIST, and then frozen, reaching a test accuracy of 0.914, 0.693, and 0.995, respectively. LoRA adapters with candidate rank set $\mathcal{R} = \{2, 4, 8, 16\}$ and provisioned width $r_{\max} = 16$ are placed on all six convolutions through 1×1 factors and on both head layers; a client assigned rank r trains with its factors sliced to the first r coordinates, so the rank genuinely bounds local adaptation capacity. Adapters initialize with A drawn from a Gaussian of standard deviation 0.1 and B at zero, so every adapter starts as the identity and all adaptation is learned in the federated phase. The CNN scale is a deliberate scope choice: it isolates the scheduling question at a payload whose full-model upload already saturates the calibrated stress deadlines. A transformer tier repeats the trainer with a tiny vision transformer (ViT): six blocks, width 192, patch 4, three heads, centrally pretrained from scratch and then frozen, with LoRA on the query–key–value, attention-output, and both multi-layer perceptron (MLP) projections of every block; its audited payload is 41,472 B per rank column (7.4× the CNN's), so its deadline ladder is calibrated separately at 2.0/0.5/0.25 s against those payloads.

Federated training hyperparameters. Federated training runs for 40 rounds with one local epoch and batch size 128. Local stochastic gradient descent (SGD) uses learning rate 0.005, weight decay 10^{-4}, gradient clipping at 1, and cosine decay. The server applies rank-masked weighted averaging with momentum $\beta = 0.3$.

Link models. Uplink rates are drawn from LogNormal $(2.5, 0.7)$ clipped to $[1, 40]$ Mbps and mobility from Beta$(2, 5)$, redrawn each round. The mobility draw is the hazard intensity μ_i^t of (3) itself: the study simulates no separate propagation, processing, handover-delay, or packet-loss process, and their contribution is absorbed into μ_i^t; the realized delivery of each update is a Bernoulli draw with probability (3) at the drawn state, so the deployed predictor is also the ground-truth generator and the estimates are oracle, a point the limitations return to. The grouped ground–aerial–satellite profile assigns 60/25/15% of clients to classes with distinct laws: ground LogNormal$(2.8, 0.5)$ clipped to $[5, 40]$ Mbps with Beta$(2, 8)$ mobility, aerial LogNormal$(2.3, 0.7)$ clipped to $[2, 40]$ Mbps with Beta$(2, 5)$, and satellite LogNormal$(1.6, 0.5)$ clipped to $[0.5, 15]$ Mbps with Beta$(2, 3)$.

Deadline calibration. All deadline regimes are calibrated to the real payloads. The nominal round deadline is 2.0 s, at which the transmission term costs only 3% of a rank-16 update's delivery probability at the median 12 Mbps rate, while a full-model update already misses 83% of its deadlines at that rate (the table entry 0.17 is the expectation over the full rate law, which the lognormal spread lifts above the median-rate factor). The stress deadlines are 0.1 s and 0.05 s, at which rank-16 timely delivery falls to 0.41 and 0.27 while rank-4 retains 0.62 and 0.57, so downshifting is genuinely load-bearing. A loose 10 s control restores full-model timely delivery to 0.52. The ViT tier uses the same construction at its own audited payloads: nominal 2.0 s, stress 0.5 s and 0.25 s.

Model and payload audit. The frozen backbone has 1.28M parameters; each adapted layer contributes a frozen-A and a trainable-B slice per rank column, and only B is transmitted. Table 2 lists the audited serialized sizes in 32-bit floating-point (FP32) alongside the 16-bit floating-point (FP16) upload format, and every deadline regime of this study is calibrated against them. The metadata term B_{meta} of (4) is zero in both evaluation tiers, since the server broadcast is not part of the uplink accounting.

Simulator. The 500-client, 150-round scheduler simulation uses the same delivery and capacity models with uniform cohort sampling (a cohort is the set of clients admitted in one round), per-client rank selection, and budget admission on the same ceiling grid; its payload table (5,632 B per rank column) is identical to the trainer's, and its MB column reports grid-billed payloads on that same ceiling (a rank-16 update is billed 0.10 MB against a true 0.090 MB), since admission and accounting share the discretization. It covers nominal, deadline, budget, mobility, and 1000-client scalability regimes, with five seeds and common traces. Its accuracy column is a bounded scheduling proxy driven by delivered useful value, not a dataset accuracy; the full-update baseline receives the same capacity credit as rank 16 under the saturating model, a disclosed modeling assumption. A

Table 2
Payload audit of real serialized update sizes. Frozen-A clients upload only the trainable B factor (1,408 parameters per rank column); the $A{+}B$ column is the joint-training equivalent. The stylized $s(r)$ column lists the protocol reused as a normalized-cost cell in the simulation study. The ceiling grid bills a rank-2 update at 0.02 MB against a true 0.011 MB (a 77% inflation, versus 11% at rank 16), a conservative bias shared by every scheduled method; the grid sweep bounds the induced error.

Rank r	$A{+}B$ FP32 (MB)	B-only FP32 (MB)	B-only FP16 (MB)	Stylized $s(r)$ (MB)
2	0.021	0.011	0.006	0.54
4	0.041	0.023	0.011	0.58
8	0.082	0.045	0.023	0.66
16	0.164	0.090	0.045	0.82
Full model	5.28 MB FP32 (5.12 backbone +0.16 adapters; 58.7× rank-16 B-only)			

separate cell repeats the earlier stylized protocol ($s(r) = 0.02r{+}0.5$ MB, rank-64 full-update stand-in, 0.25 s deadline, 12 MB budget) as a normalized-cost sensitivity check.

Scoring proxy. The simulator samples π_i uniformly from $[0.15, 0.45]$. The proxy state starts at 0.1 and updates as $a^{t+1} = a^t + \eta\hat{g}^t(1 - a^t) - \varepsilon a^t + \omega^t$, where $\hat{g}^t$ is useful delivered value normalized by K, $\eta = 0.08$, $\varepsilon = 0.0005$, and $\omega^t \sim \mathcal{N}(0, 5 \times 10^{-4})$ is a small tie-breaking noise. No per-client prior is measured in the trainer experiments, so the trainer capacity term sets $\pi_i = 1$; the sampled π_i enters only the simulator objective.

Solver study. The solver study runs 30 instances per deadline and budget cell at 500 clients, on a $q = 0.01$ MB ceiling grid. It compares the grid-based DP with value-per-byte greedy, value greedy, a cohort-myopic rule, and fixed rank 16, and adds a client-count sweep that extends to 1000 clients. Budgets are 30 MB (slack), 4 MB, and 1.2 MB; the last admits only twelve rank-16 updates on the ceiling grid of the cap $K = 25$ and is therefore binding. An independent exhaustive checker enumerates every admissible admission-and-rank profile on random small instances and confirms the DP's output profile is identical to the exhaustive optimum (60 trials, worst objective gap 4×10^{-16}; public repository).

Baselines. FedRAS is compared against fixed ranks $r = 4$ and $r = 16$, a random-rank policy, and full-model FedAvg updates (referred to as full-model updates throughout, trained at a 0.4× local learning-rate scale; at the shared 1.0× rate the head baseline loses about 0.3 points of accuracy with transient minima near 0.90, so the reduced scale is favorable to the baseline and the comparison is conservative). Three external baselines are included, all as style re-implementations under the shared protocol rather than original code: an SA-FedLoRA-style schedule that heats the rank from 2 to 16 and cools it down over the 40 rounds (the original shrinks monotonically from the full width; the heat–cool path shares its low-rank endpoint), oblivious to link state [20]; a selection baseline that samples $2K$ candidates and admits the top-K by a data-size-and-recency utility, a simplified stand-in for Oort's loss-speed product [35]; and joint A/B training with plain averaging, the aggregation-level competitor [25]. Two further network-aware rules complete the comparison: a faithful-style monotone annealing schedule that holds the full width for a short initialization phase and then shrinks monotonically to the minimum rank (the original's shrink-from-full-width path, against the heat–cool stylization), and a rate-threshold rule that assigns the full rank above an 8 Mbps estimated rate and the minimum rank below it. A delivery-only ablation (reliability-only) removes the capacity term. An offline control (FedAvg without delivery modeling) always aggregates its scheduled updates, bounding the capacity achievable without feasibility constraints. All methods share client partitions, generated link states, deadline, and seed protocol; where applicable they share the local optimizer and rank set.

Scheduling variants. Two deployment variants of the formulation are evaluated. In full grid scheduling, the aggregator solves (6) and (7) over all clients; the solver study evaluates this variant at 500-client scale, and a same-cohort variant restricts the dynamic program to a uniformly sampled cohort, isolating the value of candidate-pool widening from in-cohort rank allocation. In the real trainer, fair-cohort scheduling samples the fixed cohort of ten clients and applies the per-client rank rule $r_i^\star = \arg\max_r G_i^t(r)\hat{P}_{\mathrm{on},i}^t(r)$. A binding-budget tier tightens the trainer budget to 0.4 MB, four rank-16 grid costs, so the cap and the budget interact, and adds two budget-aware greedy arms over the full client pool (greedy by utility and greedy by utility per grid cost), so exact solving, strong heuristics, and the unchecked per-client rule meet under genuine scarcity. The variants are compared in the trainer.

Metrics. Four outcomes are reported throughout: test accuracy averaged over the final ten rounds, with the zero-communication control given by the frozen backbone on the same distribution; timely-delivery ratio, the timely deliveries of a method divided by its selected clients; bytes per round, counting attempted uplink payloads including late and failed uploads at real serialized sizes; and mean delivered rank over delivered updates. Because the scheduler assigns high ranks to clients with high delivery probability, the delivered mean rank can exceed the attempted mean implied by the byte column; this selection effect is a property of the adaptive policy, not an inconsistency (a fixed-rank policy cannot produce it). The simulator reports its bounded utility state, the mean delivery probability of admitted clients, and attempted payload totals. On the adaptive task, two further cost metrics are primary: the cumulative uplink bytes and the number of rounds needed to first reach a target accuracy, read from the same logs; one trainer run also reports the measured local-training wall time per client epoch. No sparsification, energy,

Table 3
Real-trainer deadline ladder on non-IID CIFAR-10. Nominal and 0.1 s: ten seeds; 0.05 s: five seeds. Accuracy is the final-ten-round mean (rounds 30–39, the pre-fixed terminal window); timely-delivery ratio, bytes per round (rounds 30–39, attempted uplink), and mean delivered rank follow the definitions given in the setup. Rounded cells imply $0.49/0.41 \approx 19.5\%$; the +22% headline is the ratio of unrounded seed means and the per-seed mean gain is +23% (Figure 3c). Per-seed s.d.: timely-delivery ratio 0.03–0.06, MB at most 0.05 (zero for fixed-rank policies); per-seed tables are in the public repository.

	nominal 2.0 s				deadline 0.1 s				deadline 0.05 s			
Method	**Acc.**	**Timely**	**MB**	**Rank**	**Acc.**	**Timely**	**MB**	**Rank**	**Acc.**	**Timely**	**MB**	**Rank**
FedAvg (full model)	.9135 ± .0027	0.17	52.86	—	.9153 ± .0015	0.00	52.86	—	.9147 ± .0016	0.00	52.86	—
FedLoRA $r = 4$	.9154 ± .0016	0.75	0.23	4.0	.9154 ± .0016	0.62	0.23	4	.9144 ± .0015	0.57	0.23	4
FedLoRA $r = 16$	.9153 ± .0015	0.73	0.90	16.0	.9149 ± .0019	0.41	0.90	16	.9143 ± .0020	0.27	0.90	16
Random rank	.9151 ± .0016	0.76	0.42	7.3	.9153 ± .0017	0.56	0.42	6.2	.9143 ± .0015	0.46	0.42	5.7
FedRAS (ours)	.9153 ± .0016	0.73	0.90	16.0	.9154 ± .0018	0.49	0.69	12.7	.9143 ± .0014	0.49	0.44	8.1

or fairness metric is claimed in this study; a quantization baseline (FP16 upload) is evaluated with the external baselines.

Statistical protocol. Results use five seeds (0 through 4) with common random numbers (the same underlying draws shared across methods within a seed); the CIFAR-10 nominal and 0.1 s regimes, their external baselines, and the scheduling-arm comparison at the nominal deadline use ten seeds (0 through 9). All trainer accuracies average the final ten rounds (rounds 30–39), and paired accuracy differences carry bootstrap 95% confidence intervals. FedRAS versus fixed rank 16 is tested on ten paired seeds: the timely-delivery ratio improves in ten of ten pairs (two-sided Wilcoxon signed-rank test, $p = 0.002$), by +22% as the ratio of seed means (bootstrap 95% CI $[17\%, 28\%]$); the attempted-byte saving is 23% (mean paired saving, $[21\%, 27\%]$, Wilcoxon $p = 0.002$). Accuracy equivalence uses the TOST procedure fixed in advance at a ± 0.5 percentage-point margin: the mean paired difference is +0.00 percentage points at the nominal deadline and +0.06 points under stress, with $p < 10^{-6}$ in both cases, so equivalence is established; equivalence statements for any other pair are descriptive and never rest on spread alone. Simulator comparisons and all tests beyond this pre-fixed primary comparison (FedRAS versus fixed rank 16) are secondary, uncorrected, and read as descriptive.

6.2. Main Experiment: Deadline Ladder and Delivered Capacity

The main experiment answers, in the real trainer, how the scheduler behaves as the deadline tightens: it establishes accuracy parity where the deadline is slack, then tracks every outcome down the deadline ladder and reports the ranking in the single currency of delivered update mass.

Accuracy parity at the nominal deadline. Table 3 reports ten-seed means and standard deviations after 40 federated rounds on CIFAR-10. All LoRA methods hold the backbone accuracy of 0.914: the primary pair passes the pre-fixed TOST equivalence, and the remaining spreads stay under one standard deviation, whether the rank is fixed at 4, fixed at 16, drawn at random, or selected by the rank rule. Full-parameter updates spend orders of magnitude more bytes per round (Table 2) and lose roughly three quarters of their timely deliveries because their payloads rarely fit the round deadline. The cross-dataset results in Figure 2(d) repeat the parity at 0.693 on CIFAR-100 and 0.995 on MNIST at both deadlines, with the stress picture reproduced on both datasets. At the nominal deadline the rank rule selects rank 16 for every client of every round, so FedRAS coincides with fixed rank 16 in bytes and timeliness, and in accuracy up to floating-point nondeterminism; this identity, rather than a tuned advantage, is what the nominal regime is meant to show. Figure 2(a) shows the corresponding trajectories, stable over the 40 rounds for every rank policy.

Deadline ladder and delivered-rank adaptation. With parity established at the nominal deadline, we tighten the deadline to 0.1 s and 0.05 s (Table 3) and ask where the scheduler moves. Three effects emerge. First, FedRAS downshifts under pressure. Its delivered mean rank falls from 16.0 at the nominal deadline to 12.7 at 0.1 s and 8.1 at 0.05 s, where the hazard model (3) makes rank-16 payloads marginal. Second, downshifting pays. At 0.1 s, FedRAS delivers a timely-delivery ratio of 0.49 against 0.41 for fixed rank 16 while transmitting 0.69 MB against 0.90 MB, i.e., 23% fewer bytes (Figure 3); at 0.05 s the gap widens to 0.49 against 0.27 timely deliveries at 51% fewer bytes. In the single currency of delivered update mass the ranking is sharper still: fixed rank 4 leads the timely count (0.62) but its delivered updates carry a quarter of the per-update capacity, so FedRAS delivers 42% more update mass over the terminal window (mean per-seed gain, $p = 0.002$) and 8% more than fixed rank 16 ($p = 0.02$), the most of any evaluated policy. Figure 4 shows this regime in the single currency itself: every LoRA policy rides the backbone line at parity, FedRAS along the steepest delivered-capacity path, while the full-model curve never leaves it—nothing delivered, no learning. Figure 5 assembles the entire exchange behind these numbers: the measured capacity supply, the per-class delivery price, the product peaking at $r^\star$, and the conversion of preserved capacity into accuracy as the backbone weakens. Third, full-model updates collapse completely. Their timely-delivery ratio is 0.00 at both stress deadlines, so not a single full-model update arrives on time in any seed; the stress accuracy of this row is statistically indistinguishable from the per-seed pretrained backbone (0.915 ± 0.002

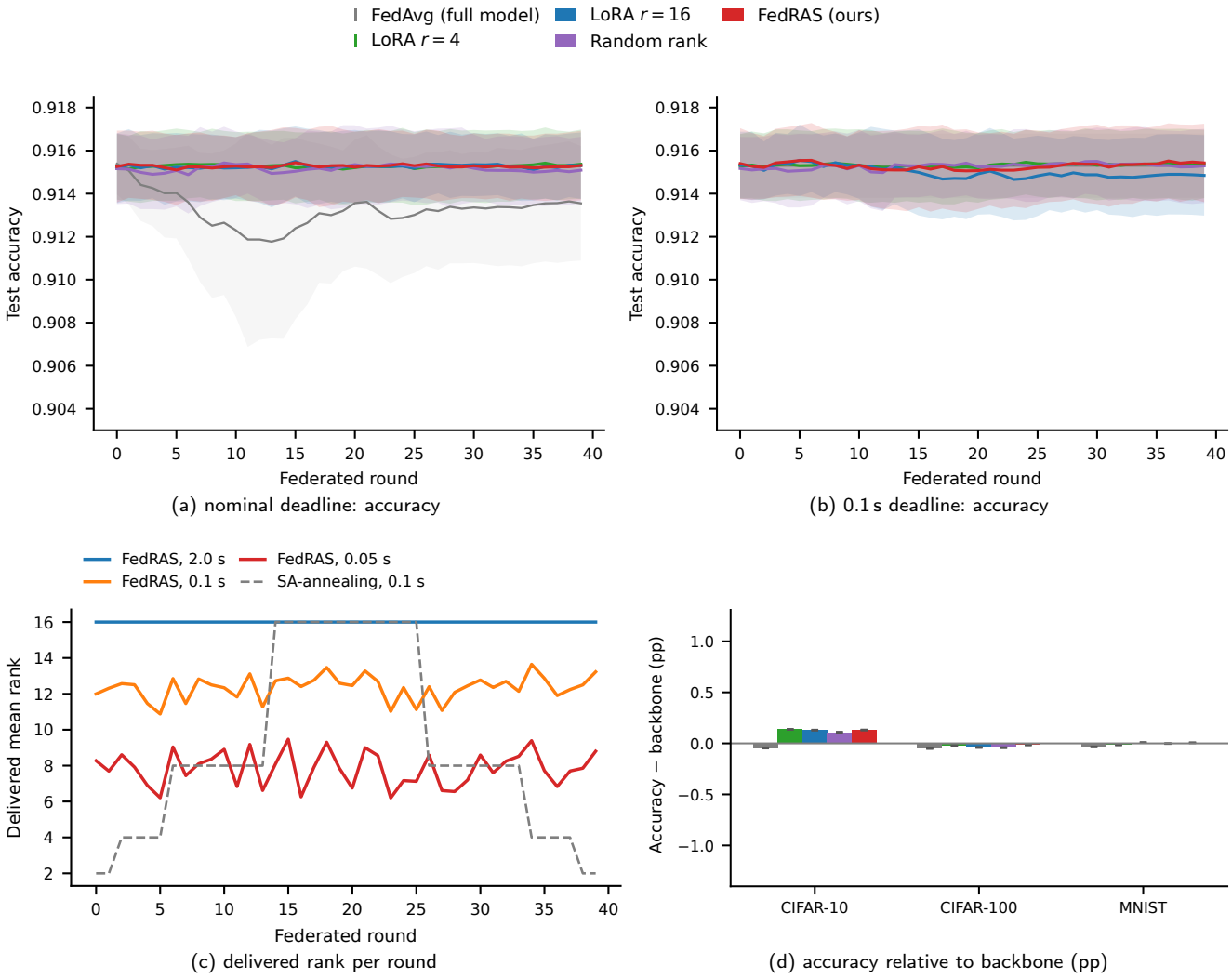


Figure 2: Real-trainer experiments (CIFAR-10, means over seeds). (a) At the nominal deadline all rank policies hold the pretrained-backbone accuracy. (b) Under the 0.1 s deadline all methods remain stable while full-model updates deliver nothing and stay at the backbone. (c) FedRAS is link-aware: delivered rank holds the provisioned width at the nominal deadline and steps down as the deadline tightens, against the fixed heat–cool path (gray dashed) that the annealing baseline follows regardless of link state. (d) Final accuracy relative to the per-dataset pretrained backbone; the parity is not CIFAR-10-specific.

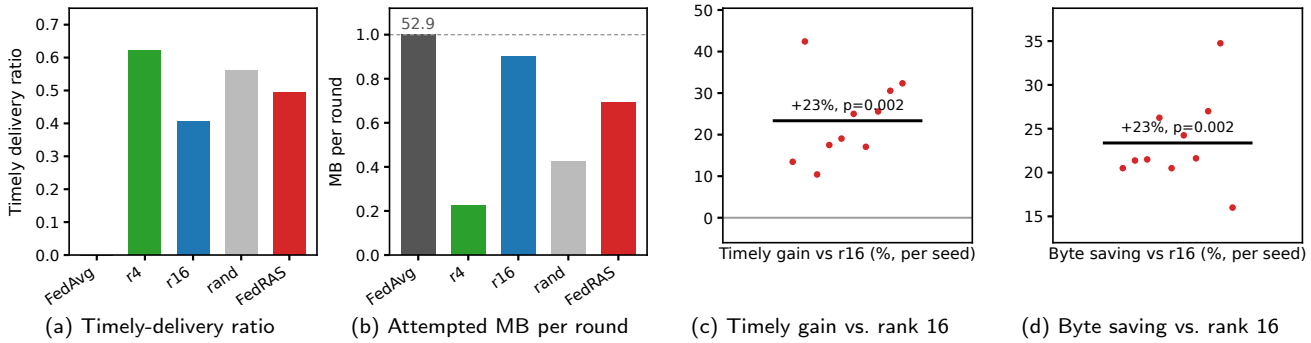


Figure 3: Deadline-stress payoff in the real trainer (CIFAR-10, 0.1 s deadline, ten seeds). (a) Timely delivery ratio; full-model updates deliver nothing. (b) Attempted uplink bytes per round; the FedAvg bar is clipped at 1.0 MB (true 52.9 MB, dashed line). (c) Per-seed relative timely-delivery gain of FedRAS over fixed rank 16: ten of ten pairs positive (per-seed mean +23%; the headline +22% is the ratio of seed means; Wilcoxon $p = 0.002$). (d) Per-seed byte saving: ten of ten pairs positive (23%, $p = 0.002$).

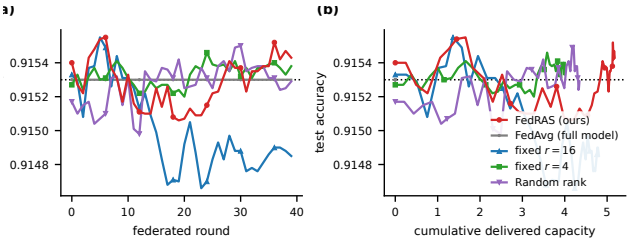


Figure 4: The strong-backbone stress regime in the single currency. (a) Accuracy trajectories and (b) accuracy against cumulative delivered update mass (terminal window, rounds 30–39, as everywhere): every LoRA policy sits at backbone parity, FedRAS reaches it along the steepest capacity path, and the flat full-model curve is the zero-communication control realized—nothing delivered, no learning.

across seeds), whose spread explains the nonzero standard deviation. This row is the zero-communication control realized, and read from the availability side it shows what the stress regime is: with nothing delivered, standard federated learning does not degrade—it performs no learning at all, the global model frozen at its broadcast state. The question in this regime is which policies keep a functioning training loop, and rank scheduling is what keeps gradient flow alive at real payload sizes; the LoRA rows meet or exceed the backbone while also moving bytes.

Figure 6 validates the closed form $r_i^\star$ of (10) against the trainer's actual assignments in the grouped-profile stress runs; the nearest-candidate reading of $r_i^\star$ reproduces the assigned rank on about three quarters of two thousand client–round points, and the remainder sits one step below at candidate boundaries, where the discrete argmax is slightly conservative.

Delivered-update norms. The capacity law is anchored in measurement: the trainer logs the mean Frobenius norm of each round's delivered updates, and across the full candidate set the measured growth tracks the $\sqrt{r}$ law of (5) (2.5× from rank 2 to 16 against 2.8× predicted, and 1.9× from rank 4 to 16 against 2.0×, with the stress series coinciding with the nominal one; Figure 7). The measured norm curve anchors the shape of (5); the exponent choice is bounded by the capacity-exponent sweep of the sensitivity study.

Why downshifting dominates. Payload grows linearly in the rank while delivery probability falls convexly once transmission time approaches the deadline, so the scheduler buys delivery probability where it is cheap and holds high rank where it is free. In this regime the binding constraint on learning is delivery, not adaptation capacity, which is why the delivered rank moves while accuracy stays at parity.

Mechanism and replication. Accuracy spreads remain small and inside the pre-fixed equivalence margin, with the primary pair passing the TOST test and further pairs read as descriptive parity; the scheduling metrics are where the methods separate (Figures 2(b) and 2(c)). All headline differences replicate across the ten paired seeds at conventional significance levels, with the statistics fixed in advance.

6.3. Adaptive Tasks and the Capacity-Conversion Law

The main experiment separated the methods in delivered adaptation capacity while leaving accuracy at parity on the strong backbone. This section locates where that capacity turns into accuracy: first on tasks whose backbone genuinely needs adaptation, then across a backbone-quality sweep that measures the conversion boundary itself.

Does communication buy learning? Two adaptive tiers answer this, each with the zero-communication control of the metrics made explicit. On the disjoint-pretraining weak backbone (backbone accuracy 0.365 on the deployment test), forty federated rounds buy roughly three points of accuracy: 0.400 for fixed rank 16 and joint A/B, 0.396 for FedRAS, 0.378 for the annealed schedule, against 0.376 for the fixed low rank (Figure 8(a)a); under the stress deadline FedRAS retains almost all of this gain while beating fixed rank 16 on timely delivery (0.49 vs 0.41) at fewer bytes. On the tiny-ViT tier with covariate-shifted clients (backbone 0.727 on the shifted test), every LoRA policy recovers about one point (FedRAS 0.735, fixed rank 16 0.737), full-model updates deliver nothing at the 0.5 s deadline and stay at the backbone, and FedRAS holds the recovered accuracy with 48% more timely updates than fixed rank 16 at 39% fewer bytes (ten of ten paired seeds, $p = 0.002$; Figure 8(b)); the right-hand panels of both figures re-plot the same trajectories against cumulative uplink bytes, the cost view the adaptive question asks for. The CNN covariate-shift tier marks the boundary of adapter recovery: all rank policies sit within half a point of the shifted backbone (0.824), so under this shift the adapters preserve rather than recover accuracy while the scheduler still wins the delivery side (0.49 vs 0.41 timely, 0.69 vs 0.90 MB). Communication pays exactly where the backbone needs adaptation, and scheduling preserves the payment under stress.

The capacity-conversion law. The strong-backbone regime makes adaptation capacity free, so preserving rank carries no measurable benefit there. To measure exactly where that stops being true, we sweep the backbone itself: one, two, three, five, ten, and thirty pretraining epochs yield measured backbone accuracies of 0.53, 0.68, 0.75, 0.82, 0.87, and 0.91 on the deployment test, and every tier runs the full six-method family with ten paired seeds at both deadlines. Two curves result (Figure 5d). The nominal coupling, fixed rank 16 over fixed rank 4, decays monotonically from +1.5 points at backbone accuracy 0.53 to zero at 0.87, and the scheduler's realized gain over the link-oblivious annealed schedule tracks it: +1.1 points at 0.53 ($p = 0.002$), +0.6 at 0.68, +0.3 at 0.75 (both $p \leq 0.05$), statistical zero from 0.82 upward—capturing about two-thirds to three-quarters of the available coupling at every tier where coupling exists. The conversion boundary is sharp and operational: *delivered capacity converts into accuracy once the backbone falls below roughly 0.8, and the conversion grows as adaptation becomes scarce.*

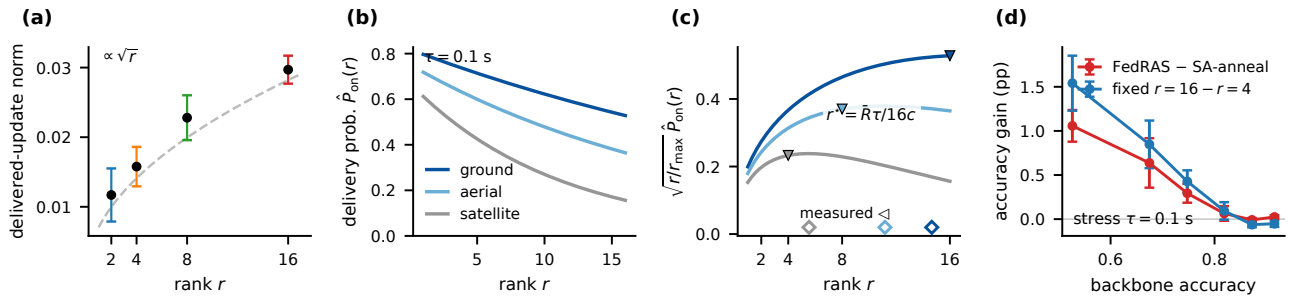


Figure 5: The capacity–delivery exchange in one picture. (a) Supply: the measured delivered-update norm grows with the square root of the rank (the anchor of (5)). (b) Price: the payload-conditioned delivery probability of (3) per SAGIN link class at the 0.1 s deadline, at each class's median rate and mean mobility. (c) Law: the product $\sqrt{r/r_{\max}}\,\hat{P}_{\text{on}}(r)$ peaks at the closed form $r^\star$ of (10) (triangles; ground peaks at the provisioned width), and the diamonds are the trainer's measured per-class delivered ranks in the grouped-profile stress runs, tracking the peaks. (d) Conversion: the accuracy value of preserved capacity as the backbone weakens (Section 6.3); the gain vanishes on the strong backbone and grows as adaptation becomes scarce.

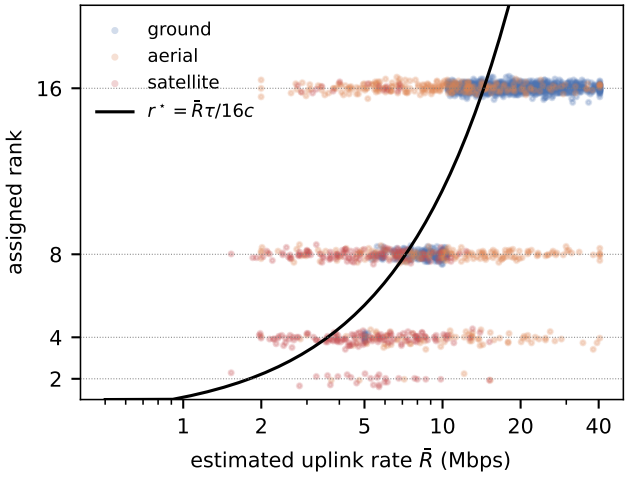


Figure 6: The closed form (10) against the trainer's actual assignments in the grouped-profile stress runs (0.1 s, five seeds, 2000 client–round points, one dot per assignment with vertical jitter). Assignments track $r_i^\star$ across all three link classes; the nearest-candidate reading of the closed form reproduces the assigned rank on about three quarters of the points, and the remainder sits one step below at candidate boundaries, where the discrete argmax is slightly conservative.

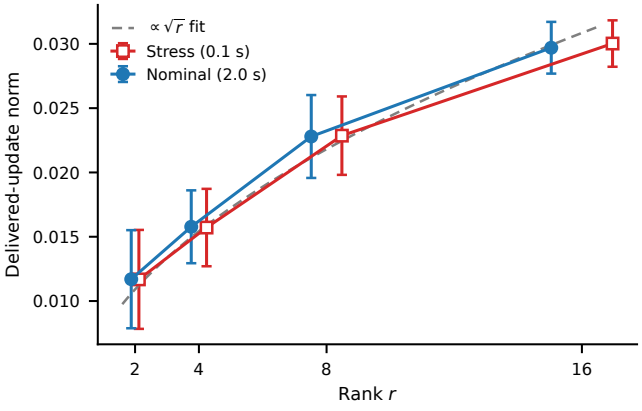


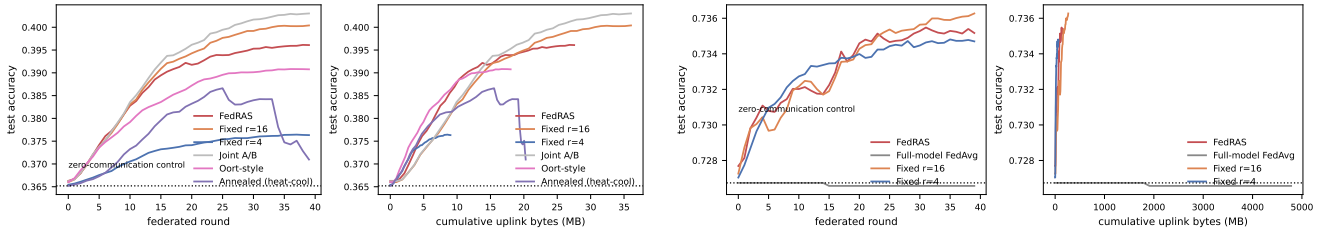
Figure 7: Delivered-update norm vs. rank on CIFAR-10 (mean over delivered rounds; five seeds per point, error bars are one standard deviation). The measured growth (2.5× from rank 2 to 16, 1.9× from rank 4 to 16) tracks the square-root capacity law of (5) (2.8× and 2.0×) rather than linear payload growth (8× and 4×); the nominal and stress series coincide.

The weakest tier in detail. At the one-epoch tier (backbone accuracy 0.534, mean over ten seeds), federated adaptation improves on the pretrained weights by roughly two points, and the rank–accuracy coupling appears at the nominal deadline (Figure 9(a)), where rank 16 reaches 0.561 against rank 4's 0.546 over ten paired seeds (+1.5 points, Wilcoxon $p = 0.002$), with FedRAS coinciding with rank 16. Under the 0.1 s deadline the coupling persists and the scheduler prices it without any retuning (Figure 9(b)): FedRAS holds a delivered rank of 12.8, finishes 1.1 points above rank 4 (ten of ten pairs), while trading 0.4 points to fixed rank 16 for a 22% higher timely-delivery ratio and 23% fewer bytes (Figure 9(d)).

No static policy is right everywhere. Table 4 assembles the four regimes the paper has measured. Each static policy is optimal in one of them—by construction or by luck of the backbone—and pays elsewhere: the low rank pays accuracy where capacity matters, the high rank pays delivery once deadlines bind, and the training-signal schedule cannot see the difference. The link-conditioned rule is the only one that is near-optimal in every regime without being told which regime it is in.

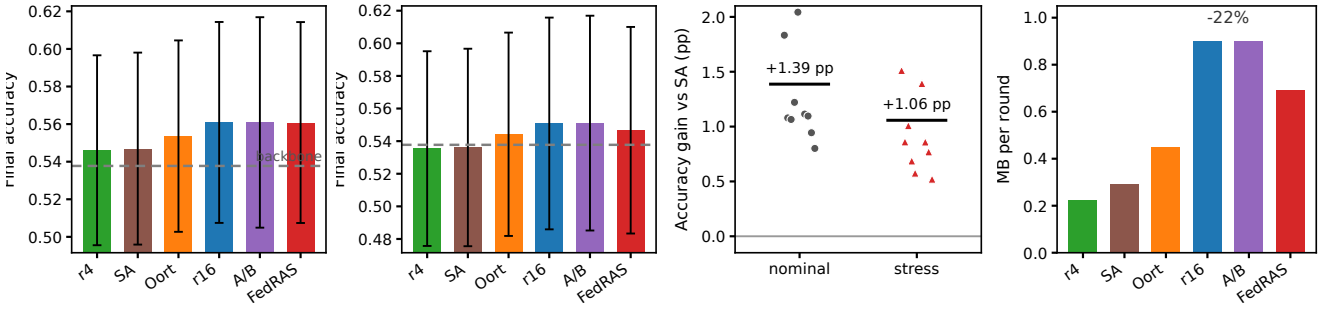
6.4. Comparison With Baselines

The comparison places FedRAS against fixed-rank and full-model policies, external schedules in both stylized and faithful variants, simple network-aware rules, and the heuristics at 500-client scale, under identical partitions, link traces, and seed protocol.



(a) weak backbone, disjoint pretraining, 0.1 s deadline (b) tiny ViT, covariate shift, 0.5 s deadline

Figure 8: Adaptive-task evidence, accuracy vs round (left of each panel) and vs cumulative uplink bytes (right); the dotted line is the zero-communication control (frozen backbone on the deployment distribution), and the flat full-model curve in (b) is that control realized: nothing delivered at the stress deadline. (a) Disjoint-pretraining weak backbone: communication buys about three points, FedRAS keeps nearly all of it at stress. (b) Shifted tiny ViT: LoRA recovers about one point, and FedRAS preserves it with +48% timely and −39% bytes against fixed rank 16 ($p = 0.002$).



(a) Nominal deadline (b) Stress deadline (0.1 s) (c) FedRAS vs. SA-anneal, per seed (d) Attempted MB per round

Figure 9: Weak-backbone capacity payoff with the literature schedules (one-epoch pretrain, ten seeds). (a) At the nominal deadline the high-rank policies lead low-rank operation by about 1.5 points and FedRAS coincides with rank 16; the dashed line marks the pretrained backbone. (b) Under the 0.1 s deadline the link-oblivious SA schedule drops onto the low-rank trajectory. (c) Paired per-seed accuracy differences against the SA schedule: every seed is positive at both deadlines (ten of ten, $p = 0.002$). (d) FedRAS transmits 23% fewer bytes than the accuracy-matching rank-16 and joint A/B policies.

Table 4
No static policy is right everywhere. Four regimes (deadline × backbone quality), four policies, ten paired seeds each; every cell reports the accuracy gap to the best policy in that regime (percentage points), the delivered update mass (terminal-window delivered capacity, % of the regime best), and the timely-delivery ratio (% of best). Fixed r=4 holds the timely lead everywhere yet pays about 1.5 points in the weak regimes and never delivers more than three quarters of the best capacity; fixed r=16 owns the nominal regimes but loses 35% of its deliveries at stress; the annealed schedule matches FedRAS on the strong backbone—low rank is adequate exactly when capacity is free—and drops 1.4–1.5 points precisely where capacity matters. FedRAS is the only policy whose worst cell is bounded at 0.41 points while holding the full delivered capacity of its regime throughout, because it reads which regime it is in from the link instead of fixing the answer offline.

Policy	nominal / strong	stress / strong	nominal / weak	stress / weak
Fixed r=4	−0.0 / 54% / 100%	−0.0 / 72% / 100%	−1.5 / 55% / 100%	−1.5 / 73% / 100%
Fixed r=16	−0.0 / 100% / 97%	−0.1 / 93% / 65%	−0.0 / 100% / 97%	−0.0 / 92% / 65%
SA anneal	−0.0 / 65% / 100%	−0.0 / 83% / 99%	−1.4 / 66% / 100%	−1.5 / 84% / 99%
FedRAS	−0.0 / 100% / 97%	−0.0 / 100% / 80%	−0.0 / 100% / 97%	−0.4 / 100% / 80%

Against low-rank baselines. Across Tables 3 and 5, the remaining policies sit on one delivery–capacity trade-off: random-rank, Oort-style, and annealed schedules deliver *more* timely updates at *fewer* bytes than FedRAS (Figure 2(c)), but at delivered ranks of 5.0–8.0 against FedRAS's 12.7. What FedRAS keeps on this trade-off is adaptation capacity. Whether preserved capacity converts into accuracy in regimes where capacity matters is tested with the weak-backbone study.

Literature-mechanism baselines. Table 5 reports the three external baselines of the setup in the real trainer. The link-oblivious annealing schedule settles at low rank and therefore buys timely delivery at the lowest byte cost among the baselines, but it cannot increase rank when links

Table 5
External baselines in the real trainer, ten seeds. The FP16 rows use the FP16 serialized sizes of Table 2. The joint A/B plain row is computed at the B-only payload for comparability; the true-payload row bills it end-to-end at the audited $A{+}B$ size (1.64 MB per ten-client round at rank 16). The near-neighbor rows (expected latency, SVD allocation, monotone annealing) are faithful-style re-implementations of the cited mechanisms under the shared protocol.

	nominal 2.0 s				deadline 0.1 s			
Method	**Acc.**	**Timely**	**MB**	**Rank**	**Acc.**	**Timely**	**MB**	**Rank**
SA-FedLoRA-style annealing	.9152 ± .0017	0.75	0.29	5.2	.9152 ± .0016	0.62	0.29	5.2
Annealing, monotone shrink	.916 ± .001	0.75	0.21	3.8	.916 ± .002	0.63	0.21	3.8
Rate threshold	.916 ± .001	0.74	0.67	11.8	.916 ± .002	0.54	0.67	11.2
Oort-style utility, fixed rank	.9150 ± .0020	0.76	0.45	8.0	.9150 ± .0019	0.54	0.45	8.0
Expected-latency rule (after ROBA/TSFA)	.915 ± .001	0.73	0.90	16.0	.915 ± .001	0.45	0.79	14.4
SVD rank allocation (after FedARA)	.915 ± .001	0.75	0.11	2.0	.915 ± .001	0.68	0.11	2.0
Joint A/B, plain averaging	.9152 ± .0016	0.73	0.90	16	.9149 ± .0019	0.40	0.90	16
Joint A/B, true $A{+}B$ payload	.915 ± .002	0.70	1.64	16	.915 ± .001	0.28	1.64	16
FedLoRA $r = 16$, FP16 upload	.9146 ± .0013	0.74	0.45	16	.9149 ± .0015	0.55	0.45	16
FedRAS, FP16 upload	.9146 ± .0011	0.74	0.45	16	.9143 ± .0017	0.62	0.35	12.5

improve. The Oort-style selector acts on selection only and leaves the rank fixed. Joint A/B training matches frozen-factor training at the adopted learning rate, and the ablations find no stability separation at an elevated learning rate either, so the case for the frozen factor rests on exactness and upload reduction rather than on an observed stability gap.

Compression baseline. A fixed rank-16 policy with an FP16 upload (two bytes per parameter, real round-trip quantization on delivery) halves the payload and, at the 0.1 s deadline, delivers 0.55 timely updates at 0.45 MB, more timely updates than any FP32 schedule at the same rank (Table 5). Stacking FedRAS on the same format extends the gain: FedRAS-FP16 beats FP16 rank 16 in ten of ten seed pairs (+12% timely, 23% fewer bytes, $p = 0.002$ each) at equal accuracy. Compression re-prices every rank; the scheduler re-optimizes the assignment under the prices that hold.

Near-neighbor mechanisms. Two literature-mechanism re-implementations complete the comparison (Table 5). The expected-latency rule (after ROBA/TSFA) reads the same estimated rates as FedRAS but admits the largest rank whose *expected* transmission time meets the deadline; at 0.1 s it holds rank 14.4 on average and delivers 0.45 at 0.79 MB, link-aware, but it prices neither loss nor admission, so it pays near-full-rank bytes for mid-rank delivery. The SVD rank allocation (after FedARA) follows the aggregated update's energy and settles at rank 2.0 with 0.68 timely at 0.11 MB, the training-side counterpart of the delivery-only policy: timely and capacity-poor. The faithful monotone annealing lands with its stylized twin (0.63 vs 0.62 timely at 0.21 MB), and the rate threshold tracks FedRAS's network columns through a fixed two-rank split. On the weak backbone every link-aware rule beats the link-oblivious schedules, expected-latency 0.565, fixed rank 16 0.551, FedRAS 0.547, against 0.536–0.544 for annealing and Oort-style, and the family result matches the positioning of Table 1: pricing the link at all is what matters, and within the link-aware family the delivery-probability pricing buys the most timely updates at the fewest bytes (FedRAS: 0.49 timely, 0.69 MB) while the expected-latency pricing retains about two points more accuracy by holding wider ranks at more bytes, the capacity–delivery trade again, now between pricing philosophies. **Against the literature schedules.** The same paired runs place the external baselines in this regime. The SA-FedLoRA-style annealing schedule sits on the low-rank trajectory and finishes 1.3 points below FedRAS at the nominal deadline and 1.1 points below under stress (ten of ten seeds each, $p = 0.002$; Figure 9(c)): its link-oblivious heat–cool path spends rounds at low rank exactly when the backbone needs capacity. The scheduler holds the rank level that delivers the accuracy; an annealing path that ended hot would recover capacity but lose the timely delivery that motivates annealing, which is the trade-off the architecture resolves. The Oort-style selector, fixed at the mid rank, lands in between, 0.7 points below FedRAS at the nominal deadline (eight of ten pairs, $p = 0.037$) and indistinguishable under stress. Joint A/B training at the full width matches the rank-16 accuracy at both deadlines; billed end-to-end at its true $A{+}B$ payload its timely-delivery ratio at 0.1 s falls to 0.28 at 1.64 MB per round (Table 5). Preserved capacity therefore converts into accuracy when the backbone is weak enough to need adaptation, and against link-oblivious rank schedules the conversion is a direct accuracy win, which is the regime in which a rank scheduler has a reason to exist.

Cross-tier summary. Taken together, the two evaluation tiers give a consistent picture. Rank choice does not measurably change accuracy for the frozen backbone at the 40-round horizon, and the headline comparisons against fixed rank 16 pass the TOST equivalence test fixed in advance at the ±0.5 percentage-point margin; an offline control that ignores delivery modeling entirely and trains on every scheduled update reaches 0.913 ± 0.002 at the nominal deadline, close to the backbone's 0.914, separating learning capacity from delivery feasibility. The methods instead separate in delivered adaptation capacity: FedRAS holds high

Table 6
Ablation study of the real trainer (nominal deadline; the rank-mask row is at 0.1 s). Seeds as in the protocol; Timely and MB follow the setup definitions and expose the behavioral effect each mechanism produces (removing the capacity term collapses the schedule to minimum rank, i.e., 0.11 MB). Min. is the minimum over rounds of the per-round seed-mean accuracy in the elevated-learning-rate check (lr 0.02).

Configuration	Accuracy	Timely	MB	Min.
Full system (ours)	.9153 ± .0015	0.73	0.90	—
w/o capacity term	.9145 ± .0015	0.75	0.11	—
w/o frozen A (joint A/B)	.9152 ± .0016	0.73	0.90	—
w/o rank mask (random ranks, 0.1 s)	.9145 ± .0016	0.58	0.42	—
w/o server momentum	.9143 ± .0012	0.74	0.90	—
frozen A, lr 0.02	.9130 ± .0012	0.74	0.90	0.913
joint A/B, lr 0.02	.9130 ± .0011	0.74	0.90	0.912

ranks on healthy links and downshifts under pressure, full-model updates collapse once deadlines bind at real payload sizes, and the exact scheduler pays through admission over the candidate pool rather than in-cohort rank allocation (Proposition 2). Two stress tests bound these claims from both sides: the delivered-capacity gains survive stacking on an FP16 codec, and a weaker backbone reveals accuracy value in the preserved capacity exactly where the strong-backbone regime shows none. These tests rest on ten-seed replications of the headline trainer regimes; they support the stated delivered-capacity comparisons and the accuracy equivalence. The discussion that follows interprets these findings, states their scope conditions, and translates them into deployment guidance.

6.5. Ablation and Sensitivity

The ablation first removes each mechanism of the architecture and then varies the solver regime, separating the value of candidate-pool widening from the value of exact solving under a binding byte budget.

Ablation analysis. Table 6 reports mechanism ablations at the nominal deadline. In the parity regime all accuracy differences sit inside the equivalence margin (descriptive for ablation pairs); the mechanisms act on scheduling behavior and aggregation correctness. Removing the capacity term collapses FedRAS to the minimum rank, the degenerate delivery-only policy that the capacity term exists to prevent; its capacity cost is invisible at nominal deadlines but material in the simulator, where reliability-only optimization loses 28 points of utility against rank 16 (Table 8(a)). Removing server momentum is accuracy-neutral here. The rank mask itself is network-metric-neutral at fixed rank policy: with the random-rank rule at the 0.1 s deadline, masked and unmasked aggregation give 0.9145 and 0.9145 final accuracy with identical byte and timeliness profiles. The mask is nonetheless what keeps heterogeneous-rank aggregation per-slice exact whenever ranks mix, as Proposition 3 states. Under the weak backbone at the 0.1 s deadline, where delivered ranks mix around 12.7 and capacity affects accuracy, masked averaging finishes 1.4 points above the unified-denominator (unmasked) rule on identical seeds (0.547 vs 0.533), and the $W_j = 0$ fallback is itself load-bearing: holding the previous server state at uncovered columns finishes another 1.7 points above the pull-to-zero default (0.564, five seeds, with half the seed variance). The aggregation choices are empirically decisive in exactly the regime the architecture targets.

Stability at elevated learning rate. Under the elevated learning rate of 0.02, the frozen-factor rule and joint A/B training are statistically indistinguishable, with final accuracies of 0.9130 and 0.9132 and transient minima of 0.913 and 0.912 across rounds and seeds. The frozen-factor rule is therefore justified by the exactness of Proposition 3 and by its single-factor uploads, not by an empirical stability margin at this scale.

Binding budget. With the trainer budget tightened to 0.4 MB, four rank-16 grid costs, so the cap and the budget interact, exact solving pays in real training: the full dynamic program delivers a timely-delivery ratio of 0.69 at 0.34 MB with delivered rank 6.0, the value greedy 0.57 at 0.36 MB (the DP wins ten of ten paired seeds, $p = 0.002$), and the value-per-byte greedy 0.78 at 0.11 MB by collapsing to rank 2.0, buying timeliness with a third of the DP's delivered capacity. The unchecked per-client rule transmits 0.69 MB, violating the budget it does not model, and fixed rank 16 delivers 0.41. Exactness therefore earns its cost precisely where the solver study predicted: under a binding budget, the admission-coupled value that per-client orderings leave behind is real, measurable, and collected by the DP.

Arms under evaluation. To attribute the value of exact scheduling, the three arms defined in the setup are compared in the real trainer, over ten seeds at the nominal deadline and five seeds under stress. The full grid arm solves the GridDP assignment over the entire 20-client population under the participation cap and a 1.0 MB budget, which equals ten rank-16 grid costs, so the budget binds only through downshifting or skipping. The same-cohort arm draws the identical cohort as the fair-cohort arm and restricts the same dynamic program to those candidates.

Where exactness pays. Table 7 yields a clean decomposition. At the nominal deadline, the same-cohort DP coincides exactly with the per-client rule, as Proposition 2 predicts; the full DP's timely-delivery gain (+9 percentage points) comes entirely from admission over the wider candidate pool. Under the 0.1 s deadline the pattern persists: the in-cohort DP matches the per-client rule, while the full DP delivers more timely updates at proportionally more bytes. The two in-cohort arms share the cohort draw and the link traces by construction, so their assignments coincide; their delivered-rank cells (12.8 vs 12.7) differ only through the independent delivery coin flips of separate runs, sampling noise on identical schedules, not a scheduling difference. Exactness therefore pays through admission control over a

Table 7
Scheduling arms in the real trainer (1.0 MB budget; the FedLoRA $r = 16$ reference is in Table 3). Nominal: ten seeds; stress 0.1 s: five seeds.

	nominal 2.0 s, ten seeds				stress 0.1 s, five seeds			
Arm	**Acc.**	**Timely**	**MB**	**Rank**	**Acc.**	**Timely**	**MB**	**Rank**
Full DP (grid assignment)	$.9151 \pm .0017$	0.82	0.90	16.0	$.9139 \pm .0009$	0.55	0.87	15.3
Same-cohort DP	$.9152 \pm .0018$	0.73	0.90	16.0	$.9139 \pm .0014$	0.51	0.70	12.8
Fair cohort (FedRAS rule)	$.9153 \pm .0015$	0.73	0.90	16.0	$.9154 \pm .0018$	0.51	0.70	12.7

Table 8
Experiments at 500-client scale (five seeds unless noted). (a) Scheduler simulation across regimes; the utility column is the bounded scheduling utility defined in the setup and must not be read as dataset accuracy, MB is the attempted uplink payload of admitted clients in the final round billed on the same 0.01 MB ceiling grid as budget admission, and FedAvg refers to full-model updates at their real serialized size. (b) Solver study with $K = 25$ and 30 instances per cell on the identical ceiling grid; the table reports heuristic objective loss relative to the grid-based DP and DP runtime.

(a) Scheduler simulation across regimes

	Nominal 2.0 s			Deadline 0.1 s			Budget 1.2 MB		
Method	**Utility**	**Timely**	**MB**	**Utility**	**Timely**	**MB**	**Utility**	**Timely**	**MB**
FedAvg (full)	.310	.34	26.5	.093	.00	26.5	.093	.00	0.0
$r = 4$	.744	.75	0.8	.689	.64	0.8	.744	.75	0.8
$r = 16$	.914	.73	2.5	.763	.42	2.5	.797	.76	1.2
Random	.816	.75	1.3	.709	.57	1.3	.803	.75	1.2
Reliability-only	.635	.76	0.5	.606	.70	0.5	.635	.76	0.5
SA-annealing	.842	.76	0.5	.730	.70	0.5	.796	.76	0.5
FedRAS (ours)	.914	.73	2.5	**.781**	.50	2.0	**.797**	.76	1.2

(b) Solver study

Deadline (s)	**Budget (MB)**	**Greedy val/byte**	**Greedy value**	**Cohort myopic**	**Fixed $r = 16$**	**DP time (ms)**
2.0	30	63.6%	0.0%	40.9%	0.0%	421
2.0	4	63.6%	0.0%	40.9%	0.0%	51
2.0	1.2	47.8%	28.4%	48.9%	28.4%	23
1.0	30	62.7%	0.0%	40.0%	0.0%	415
1.0	4	62.7%	0.0%	43.0%	0.0%	51
1.0	1.2	47.3%	29.3%	48.3%	29.3%	23
0.1	30	52.8%	0.0%	51.9%	0.0%	415
0.1	4	52.6%	0.0%	52.0%	0.0%	51
0.1	1.2	41.0%	36.0%	54.8%	36.0%	23

heterogeneous pool, not through rank allocation inside a cohort (Proposition 2). This delimits when full grid scheduling is worth its coordination cost.

Sensitivity sweeps. Figure 13 sweeps the provisioned width $r_{\max}$, the cohort size, and the label heterogeneity α on CIFAR-10 with FedRAS (five seeds per variant; the nominal reference uses ten seeds). Accuracy is flat in $r_{\max}$ and in the cohort size. Extreme label heterogeneity ($\alpha = 0.1$) reveals a second-order effect: rank-16 updates lose 0.32 percentage points relative to rank 4, consistent with capacity-amplified client drift, and full-model updates lose the most; FedRAS follows the rank-16 trajectory here, so a heterogeneity-aware downshift is a natural extension. The dominant effect of all three axes is on bytes and delivered updates (red, right axis of Figure 13).

Capacity-exponent sensitivity. The half exponent of (5) is a modeling choice anchored in the measured norms, so a solver-side sweep asks how the conclusions depend on it. Replacing $\sqrt{r/r_{\max}}$ with $(r/r_{\max})^{\gamma}$ for $\gamma \in \{0.25, 0.5, 0.75, 1\}$ and re-solving 500-client instances (five seeds per cell) at the binding 1.2 MB budget leaves every qualitative conclusion intact: value greedy loses 7–47% of the objective (least at $\gamma = 1$, where utilities linearize and value ordering packs well), value-per-byte greedy loses 0–43%, and the budget-feasible fixed-rank-16 rule loses 8–48%, while the DP's chosen mean rank stays within 7.5–8 across all exponents, the downshifting direction does not depend on the exponent. Exactness earns its cost under every sub-linear capacity law, and the measured norm curve supports precisely the sub-linear regime.

6.6. Efficiency and Robustness

The final stage asks whether the architecture is deployable: what its scheduling and local computation cost, how it behaves under heterogeneous link classes and correlated data skew, under a structurally mismatched channel, under predictor error, and across datasets and stress axes.

Measured computation cost. The measured local-training wall time is about 2.1 s per client epoch on the reference GPU under evaluation-time sharing, two orders of magnitude

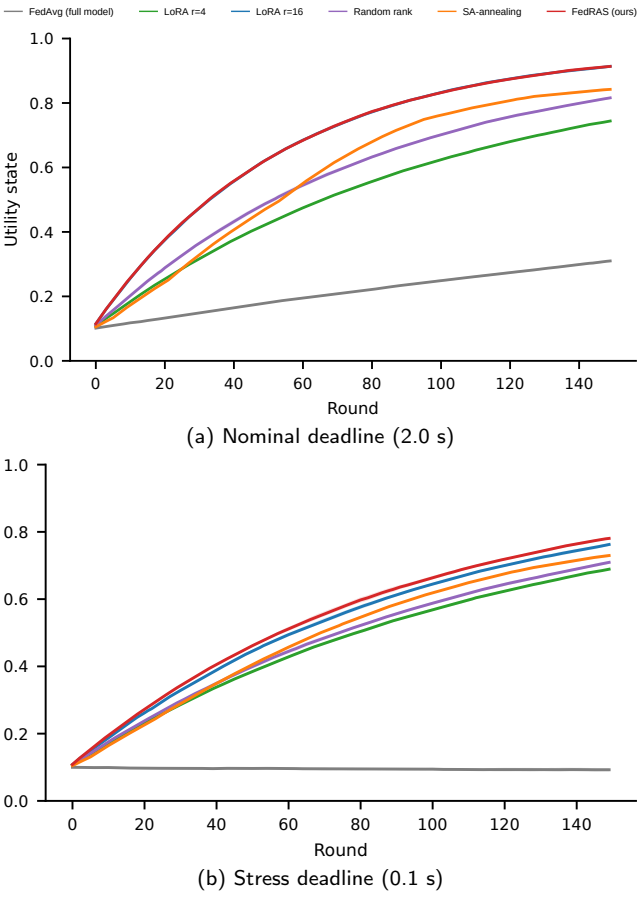


Figure 10: Scheduler simulation utility-state trajectories (mean over five seeds). (a) Nominal 2.0 s deadline, where FedRAS matches fixed rank 16. (b) 0.1 s deadline, where FedRAS downshifting separates from fixed rank 16.

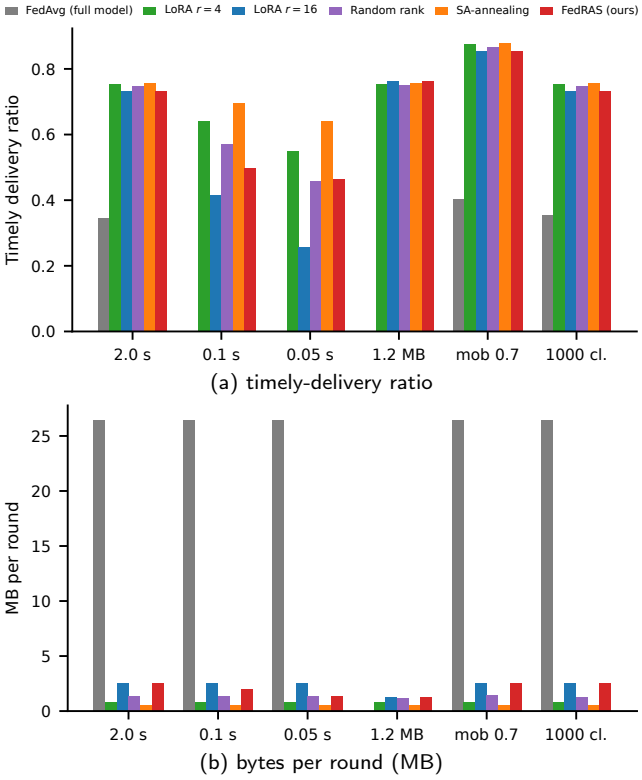


Figure 11: Timely-delivery ratio and attempted bytes per method across simulator regimes, including the 1000-client scalability regime (five-seed means, final round). Full-model updates deliver nothing under stress and cannot be admitted at all under the binding budget.

above the 0.1 s stress deadline, confirming that τ is the uplink delivery window rather than the round duration; the scheduling costs are reported with the solver study (Section 6.7).

Extended stress matrix. Table 9 extends the study along the delivery-side axes. Mobility-only scenarios leave all LoRA policies at parity at the nominal deadline with delivered ranks unchanged. By the closed form (10), the per-client rank depends on rate and deadline alone, so no mobility law can force downshifting; mobility acts on admission, not on rank. The default Beta(2, 5), a milder Beta(0.7, 5), and a harsher Beta(2, 1) with 2.3× the mean hazard all agree: under Beta(2, 1) the delivered rank stays pinned at 16 for every policy and accuracy stays at parity (0.9153/0.9156/0.9153 for rank 16/rank 4/FedRAS), while the timely-delivery ratio falls from 0.73 to 0.51. Mobility taxes delivery, never the rank choice, exactly as (10) predicts.

Loosening the deadline to 10 s restores full-model timely delivery to 0.52 without changing any LoRA accuracy.

Link-class heterogeneity and selection bias. SAGIN link profile. Grouping the 20 clients into the ground, aerial, and satellite classes of a SAGIN, each with distinct rate and mobility laws, reproduces the aggregate parity at the nominal deadline while exposing the expected per-class structure: timely delivery is 0.84 for ground, 0.75 for aerial, and 0.64 for satellite clients (terminal-window means, five seeds). At this deadline the scheduler holds rank 16 across all classes, so the class gap reflects the link laws alone rather than rank allocation. The profile shows that the scheduler responds sensibly to client-class heterogeneity. Under the 0.1 s deadline the downshifting is per class and it is precisely what protects the weakest class: FedRAS delivers a timely-delivery ratio of 0.51 in aggregate (0.57 for ground, 0.42 for aerial, and 0.43 for satellite clients) with a delivered rank of 13.1, beating fixed rank 16 (0.44 timely) at 22% fewer bytes. Against the same fixed width, whose per-class timely ratios are 0.54 (ground), 0.32 (aerial), and 0.19 (satellite), the scheduler more than doubles the satellite class's timely

Table 9
Extended stress matrix of the real trainer (round-40 accuracy, five seeds). The mobility-only and $\alpha = 1.0$ scenarios probe extreme conditions; the last rows repeat the 0.1 s deadline on CIFAR-100 and MNIST.

Scenario	FedAvg	$r = 4$	$r = 16$	Random	FedRAS
deadline 10 s	0.9134	0.9144	0.9145	0.9141	0.9143
mobility (milder)	0.9115	0.9143	0.9141	0.9145	0.9141
$\alpha = 0.1$	0.9022	0.9139	0.9107	0.9130	0.9109
$\alpha = 1.0$	0.9142	0.9143	0.9144	0.9141	0.9144

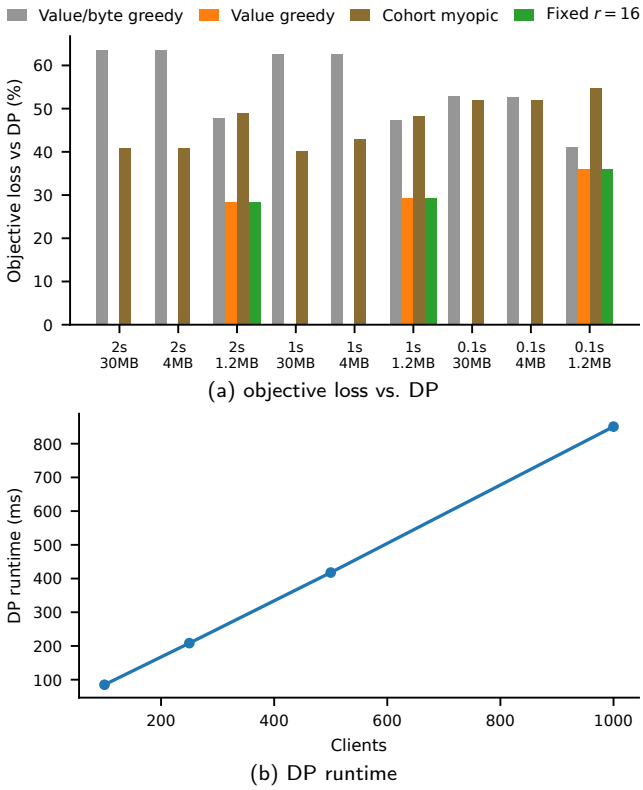


Figure 12: Solver study. (a) Objective loss of the heuristics relative to the DP across deadline and budget cells. (b) DP runtime scaling with the client population.

delivery while the ground class—held at the full width—loses nothing: the class gap that the link laws impose on a fixed payload is largely closed by letting the rank move with the link.

Correlated link–data skew. Correlating this class structure with the data sharpens the question: when the weak-link classes live exclusively on the aerial and satellite clients, the participation bias that no aggregation rule can remove is measurable but bounded. At the 0.1 s deadline the weak-link half of the classes trails the strong-link half by 4.9 points under FedRAS, against 5.8 under fixed rank 16 and 3.2 under fixed rank 4; FedRAS sits within a point and a half of the low-rank fairness floor while holding three times its delivered capacity, and its aggregate accuracy (0.9125) stays within the equivalence band. The bias is a property of link-dependent participation, not of the rank rule; charging it fully is what per-client fairness accounting would require, and it composes with the scheduler as an additional constraint rather than a redesign.

De-circularized delivery arm. The headline evaluation generates realized delivery from the same hazard model the scheduler optimizes, with oracle link states. To test that the delivered-capacity gains are not an artifact of this closed loop, a dedicated arm replaces both. Realized rates follow a per-client first-order autoregressive (AR(1)) log-process with autocorrelation 0.6 and stationary marginal equal to the headline LogNormal(2.5, 0.7) law, so the history-based estimate, an exponential moving average (EMA) with coefficient 0.5 over past realized rates, exactly the estimation step of Algorithm 1, is stale. Realized delivery is structural: a propagation delay of 5 ms, a handover event with probability 0.15 adding 20 ms, and a loss event with probability $1 - e^{-1.5\mu_i^t}$, rather than a draw from the hazard model. The arm runs at the 0.1 s stress deadline. The scheduler plans against the stale EMA estimate and the mobility intensity alone; delivery succeeds iff the structural delivery time (propagation, handover, and transmission at the realized rate) meets the deadline and no loss event occurs.

Results. Under the structural channel the ordering of the hazard-model arm survives with stale, noisy estimates: FedRAS delivers a timely-delivery ratio of 0.59 at 0.73 MB with delivered rank 13.1, against 0.52 at 0.90 MB for fixed rank 16, the capacity-matched comparator, while the fixed low rank keeps the timely lead (0.67 at 0.23 MB) at a third of the delivered capacity, and full-model updates deliver nothing. Accuracy stays at parity (0.915–0.916) throughout, as expected with the strong backbone. The delivered-capacity gains therefore survive a structurally mismatched channel in both direction and mechanism—buying delivery where it is cheap by shedding rank and preserving capacity where it pays—and the magnitudes soften relative to the oracle arm, which is the honest price of the closed-loop assumption the headline tiers carry.

Delivery-predictor sensitivity.

The calibration study of Figure 14 measures both the fidelity of the deployed predictor and the scheduling cost of its errors (500 clients). With the oracle link state of the setup, the hazard predictor (3) is calibrated by construction. Deliberately distorted variants, power distortions of the predicted probability and multiplicative lognormal noise, are measurably miscalibrated, yet the resulting assignment-objective loss, evaluated under the true probabilities, stays below 1% on average in every cell. The loss follows the structure of the error: power distortions cost most under the stress deadline, where rank ordering matters most, while noise costs most in the nominal slack cell, where it perturbs an otherwise near-uniform rank preference. Mis-prediction therefore degrades the objective smoothly rather than catastrophically, and the observed delivery outcomes of Algorithm 1 are exactly the signal from which a per-client predictor would be recalibrated.

6.7. Scale and Solver Study

The simulation tier asks whether the trainer conclusions survive an order of magnitude more clients, and the solver study asks what exactness costs and returns at that scale.

Scheduler simulation across regimes. Table 8(a) summarizes the simulator across regimes at the final round. In the nominal regime, FedRAS matches fixed rank 16 and dominates every low-rank policy. Under the 0.1 s deadline, FedRAS leads fixed rank 16 at 20% fewer bytes (2.0 versus 2.5 MB), and full-model updates deliver nothing. Under the binding 1.2 MB budget, full-model updates cannot be admitted even once, while FedRAS and rank 16 tie. The

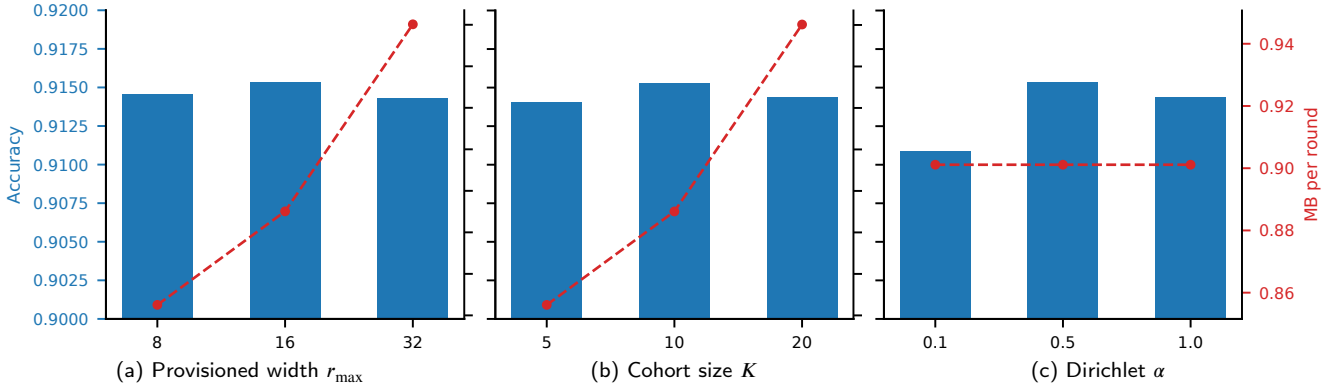


Figure 13: Sensitivity of the trainer to provisioned adapter width, cohort size, and label heterogeneity on CIFAR-10 with FedRAS (five seeds per variant; the nominal reference column uses ten seeds). Accuracy is flat in $r_{\max}$ and cohort size, with a second-order drop only at $\alpha = 0.1$; the dominant effect of all three axes is on bytes and delivered updates (red, right axis).

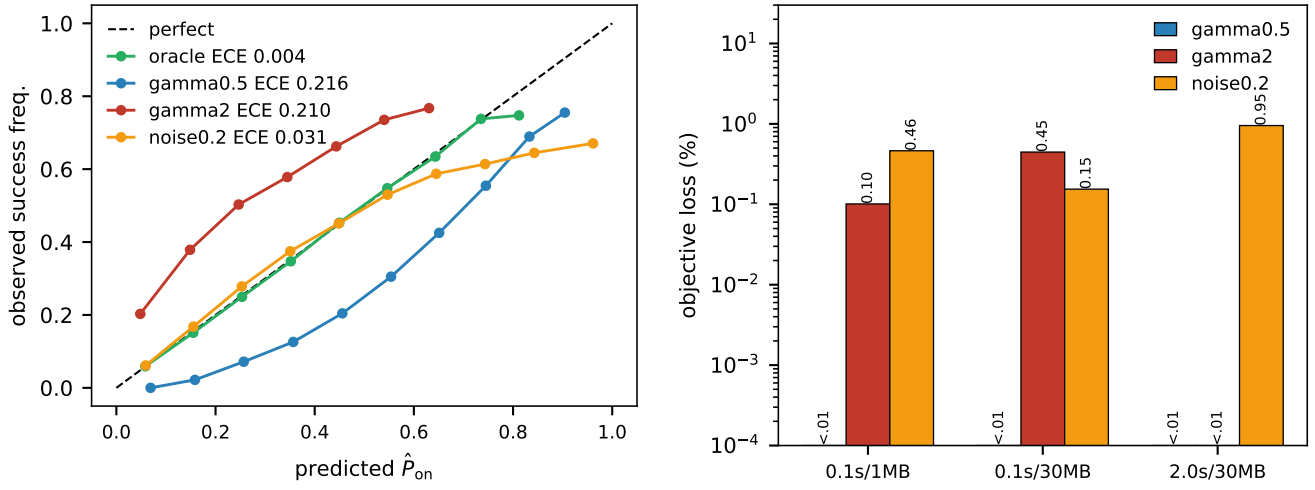


Figure 14: Delivery-predictor calibration study (500 clients, 150 round-instances per cell). (a) Reliability at the 0.1 s deadline; with oracle link state the hazard predictor is calibrated by construction, while the power-distorted (γ) and noisy variants are measurably miscalibrated. (b) Assignment-objective loss of optimizing against the distorted predictor, evaluated under true probabilities, at most 1% on average in every cell; the $\gamma = 0.5$ loss is $< 0.001\%$ throughout, labeled explicitly on the log-scaled panel.

mobility-stress and 1000-client regimes show the same pattern: FedRAS and fixed rank 16 coincide at the top of the utility column, so scheduling separates only where deadlines or budgets bind. A final cell repeats the stylized protocol of the earlier literature: with the $s(r)$ payload model the ordering is preserved, confirming that the conclusions are not an artifact of the payload accounting.

Where the simulator gain comes from. Fixed rank 16 spends uniformly, weak-link clients miss deadlines while strong-link clients are under-served, whereas FedRAS reallocates, holding high ranks where delivery survives them and downshifting or skipping the rest. Reliability-only optimization fails the other way, buying timely delivery with capacity. The dynamic program between these extremes prices capacity and delivery in a common unit, and the grid makes the resulting knapsack exactly solvable.

Trajectories. Figure 10 shows the nominal and 0.1 s trajectories, and Figure 11 the delivery-side view across regimes.

Solver study: heuristic gaps and runtime. Table 8(b) reports the scheduling study with the identical ceiling discretization for the DP and every heuristic. When the byte budget is slack, value greedy and the fixed rank-16 rule are already optimal, so the choice of scheduler matters little; the value-per-byte greedy, by contrast, collapses to a smallest-rank preference and already loses up to 63.6% of the objective in the slack cells. The interesting regimes are the binding ones: once the budget binds, every heuristic loses a substantial fraction of the objective, up to 54.8% for the cohort-myopic rule, while the DP reaches the optimum of the discretized single-round estimated problem in every cell. Its runtime is sub-second at 500 clients and scales linearly

to 1000 (Figure 12); scheduling precedes the uplink window and is charged to round setup rather than τ (23–421 ms per round for the DP across the evaluated grid widths; sub-millisecond decisions for the per-client rule).

Why the heuristics fail where they do. The failure modes are structural rather than numerical, and both were already visible in the models. Value-per-byte pricing divides a capacity gain that saturates with rank by a payload that grows linearly in (4), so its ratio favors the smallest rank regardless of link state. Fixed rank 16 and value greedy fail only under scarcity, where choosing which candidates to drop is precisely the admission coupling that per-client rules cannot resolve. The DP runtime grows with the client count but is far more sensitive to the budget grid width, which is why it is fastest under exactly the binding budgets that need it most.

Grid sweep. A dedicated grid sweep around the default step bounds the quantization error at about 3% in the binding regime and below 1% in slack regimes; a finer step reduces it further at a known runtime cost. The grid exactness claim therefore applies to a grid whose quantization error is bounded, measured, and adjustable.

7. Discussion and Limitations

7.1. Interpretation and Deployment

Why the strong backbone licenses network control of the rank. With a strong frozen backbone, transmitted rank does not measurably change test accuracy. This invariance is the enabling condition of the architecture rather than a weakness of the results: precisely because accuracy does not depend on the rank, the rank is free to be set by the link, and the scheduler's advantage appears as delivered adaptation capacity, bytes, and energy rather than as accuracy points—the contribution currency of communication-efficiency and client-selection systems, here evaluated at non-degraded accuracy. When the backbone is weak enough to need adaptation, the preserved capacity converts into accuracy along the measured backbone-quality sweep. When the pool fits the cap and the budget accommodates every candidate at $r_{\max}$, the per-client rule is already optimal (Proposition 2) at zero overhead, and exact scheduling pays only when the budget binds or the pool must be culled. Under extreme label heterogeneity high ranks lose accuracy through capacity-amplified drift, and under tight deadlines the weakest client class still delivers least.

Deployment. Calibrate the deadline ladder from the real payload of the widest rank, size the byte budget at ten times the grid cost of $r_{\max}$, use Proposition 2 as a runtime switch, and treat the predictor as a monitored plug-in, since distortions cost at most about 1% of the objective (Figure 14). The delivered-capacity advantage translates directly into operator currencies: under the airtime-proportional transmit-energy model, the stress configuration delivers about 41% more adaptation capacity per joule than fixed rank 16, whose airtime is 59% spent on updates that never arrive, and matching FedRAS's timely ratio would require about 1.5× the uplink rate for the fixed-rank fleet. FedRAS is an aggregator-side module that leaves client training untouched.

7.2. Limitations

The results are gains in delivered adaptation capacity in the evaluated setting and, where the backbone needs adaptation, in accuracy—not universal accuracy gains—and centralized pretraining is a standing premise of parameter-efficient FL. The hazard model (3) assumes the closed-form realization with oracle link states, which the de-circularized arm probes but does not remove, and rates are drawn independently, so the shared-spectrum case is untested. The $\sqrt{r}$ capacity law assumes the delivered-update norm is a faithful proxy for contribution, an assumption the exponent sweep bounds. Conclusions are horizon-bound to forty rounds on a star topology, and the CNN covariate-shift tier marks a task boundary where adapters preserve rather than recover accuracy. This work proposes no protection mechanism and makes no privacy claim, and differential-privacy accounting, secure aggregation, and Byzantine robustness compose orthogonally with rank scheduling [43, 44].

7.3. Future Work

The immediate extensions are horizon-aware scheduling that carries budgets across rounds and exploits mobility correlation, per-client predictors calibrated on measured wireless traces, the correlated-spectrum case where clients share a cell or beam, heterogeneity-aware capacity terms, and composition with privacy accounting. Transformer backbones beyond the tiny ViT tier, wider rank sets, and validation on operational mobile-edge networks would further test where rank scheduling pays.

8. Conclusion

FedRAS treats the LoRA rank as a schedulable network resource accounted at its real serialized size and runs a closed round loop in which client–rank pairs are priced by the payload-conditioned hazard model and a saturating capacity gain, admission and rank are decided jointly by an exact grid dynamic program with a provably optimal cheap fallback, and heterogeneous-rank updates are merged by frozen-factor masked averaging. The formulation yields the closed form $r_i^\star = \bar{R}_i^t \tau/(16c)$ and a one-pass check that invokes exact solving only where scarcity couples decisions.

In the single currency of delivered update mass, FedRAS delivers the most of any evaluated policy under the 0.1 s stress deadline—22% more timely updates than the capacity-matched fixed rank 16 at 23% fewer bytes and 42% more update mass than the timely-count leader fixed rank 4 (ten of ten paired seeds, $p = 0.002$)—at statistically equivalent accuracy, while full-model updates miss every stress deadline and the loop delivers nothing. A six-point backbone-quality sweep locates the conversion boundary: delivered capacity turns into accuracy once the backbone falls below roughly 0.8, reaching +1.1 points over link-oblivious schedules at backbone accuracy 0.53, one point under covariate shift on the transformer tier, and three points

over the zero-communication control with disjoint pretraining. Under a binding 0.4 MB budget the dynamic program beats the strongest value greedy by twelve timely points at fewer bytes, and every link-aware rule, including reimplementations after ROBA/TSFA and FedARA, beats the link-oblivious schedules on the weak-backbone task.

The gains come from the pricing rather than the solver's arithmetic, and exactness is free in slack regimes through the runtime switch and decisive where budgets bind. The open bounds are the independent-generation assumption on link laws, the bounded capacity exponent, the covariate-shift boundary, and the forty-round horizon. The next steps are horizon-aware scheduling, trace-calibrated predictors, correlated spectrum, and deployment validation, all governed by the condition FedRAS established, that the delivery probability of the payload, not the expectation of its delay, should set the rank.

CRediT authorship contribution statement

Wenyu Ma: Methodology, Software, Validation, Investigation, Formal analysis, Data curation, Visualization, Writing – original draft. **Xinlei Liu:** Conceptualization, Funding acquisition, Resources, Project administration, Supervision, Writing – review & editing. **Tao Hu:** Conceptualization, Funding acquisition, Resources, Project administration, Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This work was supported in part by the National Key Research and Development Program of China (No. 2024YFB2907201, No. 2024YFB2907202).

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used GLM-5.3 in order to improve the language and readability of the manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Data availability

The complete source code (trainer, simulator, solver, and analysis scripts) and the result data backing every table and figure, per-run configuration files, per-seed result tables, and raw training logs, are publicly available at https://github.com/Jude-max11/FedRAS.

References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, B. A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS), 2017, pp. 1273–1282.

[2] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, R. G. L. D'Oliveira, H. Eichner, S. E. Rouayheb, D. Evans, J. Gardner, Z. Garrett, A. Gascón, B. Ghazi, P. B. Gibbons, M. Gruteser, Z. Harchaoui, C. He, L. He, Z. Huo, B. Hutchinson, J. Hsu, M. Jaggi, T. Javidi, G. Joshi, M. Khodak, J. Konečný, A. Korolova, F. Koushanfar, S. Koyejo, T. Lepoint, Y. Liu, P. Mittal, M. Mohri, R. Nock, A. Özgür, R. Pagh, M. Raykova, H. Qi, D. Ramage, R. Raskar, D. Song, W. Song, S. U. Stich, Z. Sun, A. T. Suresh, F. Tramèr, P. Vepakomma, J. Wang, L. Xiong, Z. Xu, Q. Yang, F. X. Yu, H. Yu, S. Zhao, Advances and open problems in federated learning, Foundations and Trends in Machine Learning 14 (1–2) (2021) 1–210. doi:10.1561/2200000083.

[3] D. C. Nguyen, M. Ding, P. N. Pathirana, A. Seneviratne, J. Li, H. V. Poor, Federated learning for internet of things: A comprehensive survey, IEEE Communications Surveys & Tutorials 23 (3) (2021) 1622–1658. doi:10.1109/COMST.2021.3075439.

[4] J. Liu, Y. Shi, Z. M. Fadlullah, N. Kato, Space-air-ground integrated network: A survey, IEEE Communications Surveys & Tutorials 20 (4) (2018) 2714–2741. doi:10.1109/COMST.2018.2841996.

[5] S. Mahboob, L. Liu, Revolutionizing future connectivity: A contemporary survey on ai-empowered satellite-based non-terrestrial networks in 6g, IEEE Communications Surveys & Tutorials 26 (2) (2024) 1279–1321. doi:10.1109/COMST.2023.3347145.

[6] R. Hamdi, M. Chen, A. Ben Said, M. Qaraqe, H. V. Poor, Federated learning over energy harvesting wireless networks, IEEE Internet of Things Journal 9 (1) (2022) 92–103. doi:10.1109/JIOT.2021.3089054.

[7] N. Razmi, B. Matthiesen, A. Dekorsy, P. Popovski, Ground-assisted federated learning in leo satellite constellations, IEEE Wireless Communications Letters 11 (4) (2022) 717–721. doi:10.1109/LWC.2022.3141120.

[8] F. Tang, C. Wen, X. Chen, N. Kato, Federated learning for intelligent transmission with space-air-ground integrated network toward 6g, IEEE Network 37 (2) (2023) 198–204. doi:10.1109/MNET.104.2100615.

[9] Y. Qin, Y. Yang, F. Tang, X. Yao, M. Zhao, N. Kato, Differentiated federated reinforcement learning based traffic offloading on space-air-ground integrated networks, IEEE Transactions on Mobile Computing 23 (12) (2024) 11000–11013. doi:10.1109/TMC.2024.3389011.

[10] M. Ribero, H. Vikalo, G. de Veciana, Federated learning under intermittent client availability and time-varying communication constraints, IEEE Journal of Selected Topics in Signal Processing 17 (1) (2023) 98–111. doi:10.1109/JSTSP.2022.3224590.

[11] T. Nishio, R. Yonetani, Client selection for federated learning with heterogeneous resources in mobile edge, in: IEEE International Conference on Communications (ICC), 2019. doi:10.1109/ICC.2019.8761315.

[12] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, LoRA: Low-rank adaptation of large language models, in: International Conference on Learning Representations (ICLR), 2022.

[13] S. Babakniya, A. R. Elkordy, Y. H. Ezzeldin, Q. Liu, K.-B. Song, M. El-Khamy, S. Avestimehr, SLoRA: Federated parameter efficient fine-tuning of language models, in: International Workshop on Federated Learning in the Age of Foundation Models (FedLLM), 2023, arXiv:2308.06522.

[14] J. Zhang, S. Vahidian, M. Kuo, C. Li, R. Zhang, T. Yu, Y. Zhou, G. Wang, Y. Chen, Towards building the federated GPT: Federated instruction tuning, in: IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2024, pp. 6915–6919. doi:10.1109/ICASSP48485.2024.10447454.

[15] Y. Sun, Z. Li, Y. Li, B. Ding, Improving LoRA in privacy-preserving federated learning, in: International Conference on Learning Representations (ICLR), 2024.

[16] Z. Zhang, Y. Yang, Y. Dai, Q. Wang, Y. Yu, L. Qu, Z. Xu, FedPETuning: When federated learning meets the parameter-efficient tuning methods of pre-trained language models, in: Findings of the Association for Computational Linguistics: ACL 2023, 2023, pp. 9963–9977. doi:10.18653/v1/2023.findings-acl.632.

[17] Y. J. Cho, L. Liu, Z. Xu, A. Fahrezi, G. Joshi, Heterogeneous LoRA for federated fine-tuning of on-device foundation models, in: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2024, pp. 12903–12913. doi:10.18653/v1/2024.emnlp-main.717.

[18] B. Fan, X. Su, S. Tarkoma, P. Hui, HeLoRA: LoRA-heterogeneous federated fine-tuning for foundation models, ACM Transactions on Internet Technology 25 (2) (2025) 11:1–11:22. doi:10.1145/3723877.

[19] F. Wu, J. Hu, G. Min, S. Wang, Adaptive rank allocation for federated parameter-efficient fine-tuning of language models, IEEE Transactions on Computers 75 (4) (2026) 1650–1663. doi:10.1109/TC.2026.3655161.

[20] Y. Yang, X. Liu, T. Gao, X. Xu, G. Wang, SA-FedLoRA: Adaptive parameter allocation for efficient federated learning with LoRA tuning, arXiv preprint arXiv:2405.09394v1 (2024).

[21] L. Yi, H. Zhang, H. Yu, G. Wang, X. Liu, Q. Hu, pFedLoRA: Model-heterogeneous personalized federated learning with homogeneous low-rank adapter sharing on mobile edge devices, IEEE Transactions on Mobile Computing 25 (9) (2026) 13898–13911. doi:10.1109/TMC.2026.3674996.

[22] Y. Yang, G. Long, T. Shen, J. Jiang, M. Blumenstein, Dual-personalizing adapter for federated foundation models, in: Advances in Neural Information Processing Systems 37 (NeurIPS), 2024.

[23] Z. Wang, Z. Shen, Y. He, G. Sun, H. Wang, L. Lyu, A. Li, FLoRA: Federated fine-tuning large language models with heterogeneous low-rank adaptations, in: Advances in Neural Information Processing Systems (NeurIPS), 2024.

[24] Y. Byun, J. Lee, Towards federated low-rank adaptation of language models with rank heterogeneity, in: Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL), Short Papers, 2025.

[25] P. Guo, S. Zeng, Y. Wang, H. Fan, F. Wang, L. Qu, Selective aggregation for low-rank adaptation in federated learning, in: International Conference on Learning Representations (ICLR), 2025.

[26] Y. Zou, Z. Wang, Y. Shi, Y. Zhou, Joint rank optimization and bandwidth allocation for heterogeneous federated LoRA fine-tuning, IEEE Transactions on Vehicular Technology (2025). doi:10.1109/TVT.2025.3623104.

[27] B. Kim, W. Choi, Two stage wireless federated LoRA fine-tuning with sparsified orthogonal updates (2025). arXiv:2505.00333.

[28] Y. Hou, J. Geng, B. Li, X. Tao, J. Wang, X. Xu, B. Luo, Adaptive federated LoRA in heterogeneous wireless networks with independent sampling (2025). arXiv:2505.23555.

[29] Y. Lin, S. Han, H. Mao, Y. Wang, W. J. Dally, Deep gradient compression: Reducing the communication bandwidth for distributed training, in: International Conference on Learning Representations, 2018.

[30] H. Liu, F. He, G. Cao, Communication-efficient federated learning for heterogeneous edge devices based on adaptive gradient quantization, in: Proceedings of IEEE INFOCOM, 2023, pp. 1–10. doi:10.1109/INFOCOM53939.2023.10228970.

[31] A. Reisizadeh, A. Mokhtari, H. Hassani, A. Jadbabaie, R. Pedarsani, Fedpaq: A communication-efficient federated learning method with periodic averaging and quantization, in: Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics, Vol. 108 of Proceedings of Machine Learning Research, PMLR, 2020, pp. 2021–2031.

[32] Y. Oh, N. Lee, Y.-S. Jeon, H. V. Poor, Communication-efficient federated learning via quantized compressed sensing, IEEE Transactions on Wireless Communications 22 (2) (2023) 1087–1100. doi:10.1109/TWC.2022.3201207.

[33] Y. Oh, Y.-S. Jeon, M. Chen, W. Saad, Fedvqcs: Federated learning via vector quantized compressed sensing, IEEE Transactions on Wireless Communications 23 (3) (2024) 1755–1770. doi:10.1109/TWC.2023.3291877.

[34] M. Kim, A. L. Swindlehurst, D. Park, Beamforming vector design and device selection in over-the-air federated learning, IEEE Transactions on Wireless Communications 22 (11) (2023) 7464–7477. doi:10.1109/TWC.2023.3251339.

[35] F. Lai, X. Zhu, H. V. Madhyastha, M. Chowdhury, Oort: Efficient federated learning via guided participant selection, in: Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2021, pp. 19–35.

[36] Z. Chai, A. Ali, S. Zawad, S. Truex, A. Anwar, N. Baracaldo, Y. Zhou, H. Ludwig, F. Yan, Y. Cheng, TiFL: A tier-based federated learning system, in: Proceedings of the 29th International Symposium on High-Performance Parallel and Distributed Computing (HPDC), 2020. doi:10.1145/3369583.3392686.

[37] X. Chen, X. Zhou, H. Zhang, M. Sun, H. V. Poor, Client selection for wireless federated learning with data and latency heterogeneity, IEEE Internet of Things Journal 11 (19) (2024) 32183–32196. doi:10.1109/JIOT.2024.3425757.

[38] L. Dong, Y. Zhou, L. Liu, Y. Qi, Y. Zhang, Age of information based client selection for wireless federated learning with diversified learning capabilities, IEEE Transactions on Mobile Computing 23 (12) (2024) 14934–14945. doi:10.1109/TMC.2024.3450549.

[39] B. Luo, X. Li, S. Wang, J. Huang, L. Tassiulas, Cost-effective federated learning in mobile edge networks, IEEE Journal on Selected Areas in Communications 39 (12) (2021) 3606–3621. doi:10.1109/JSAC.2021.3118436.

[40] D.-J. Han, W. Fang, S. Hosseinalipour, M. Chiang, C. G. Brinton, Orchestrating federated learning in space-air-ground integrated networks: Adaptive data offloading and seamless handover, IEEE Journal on Selected Areas in Communications 42 (12) (2024) 3505–3520. doi:10.1109/JSAC.2024.3459090.

[41] L. Yu, X. Sun, R. Albelaihi, C. Yi, Latency-aware semi-synchronous client selection and model aggregation for wireless federated learning, Future Internet 15 (11) (2023) 352. doi:10.3390/fi15110352.

[42] Z. Lin, Z. Chen, Z. Fang, X. Chen, X. Wang, Y. Gao, FedSN: A federated learning framework over heterogeneous LEO satellite networks, IEEE Transactions on Mobile Computing 24 (3) (2025) 1293–1307. doi:10.1109/TMC.2024.3481275.

[43] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, K. Seth, Practical secure aggregation for privacy-preserving machine learning, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS), 2017, pp. 1175–1191. doi:10.1145/3133956.3133982.

[44] K. Wei, J. Li, C. Ma, M. Ding, W. Chen, J. Wu, M. Tao, H. V. Poor, Personalized federated learning with differential privacy and convergence guarantee, IEEE Transactions on Information Forensics and Security 18 (2023) 4488–4503. doi:10.1109/TIFS.2023.3293417.